

Atomic Swaps with Relaxed Assumptions

Paul Rösler , Katharina Schambach and Julian Thomas^a 

Friedrich-Alexander-Universität Erlangen-Nürnberg, Chair of Applied Cryptography, Erlangen, Germany

Abstract. Atomic swaps allow two parties to exchange coins across blockchains without trust. Most existing solutions support a 1-to-1 transaction; however, the value ratio of two cryptocurrencies is usually not 1-to-1. In addition, these solutions often rely on blockchains with explicit support structures for atomic swaps, typically hashed timelocks (HTLCs) or smart-contract scripting. Thyagarajan et al. (SP’22) provide a solution for n-to-m swaps utilizing adaptor signatures instead, which offer better privacy for the participants. Significantly, all these approaches need timed transactions or a similar substitute on both sides, but not every blockchain supports timed transactions. Hoenisch et al. (ESORICS’22) demonstrate a concrete swap procedure between Bitcoin and Monero, where only one side needs timed transactions. We build upon both of these ideas to provide a generic solution for any blockchain, where only one side needs timed transactions.

We introduce a generic adaptor-signature protocol for n-to-m swaps. In doing so, we minimize assumptions compared to current solutions. Both sides lock their coins in shared (two-party signature) addresses, where they can only access the coins together. To prevent deadlocks, both parties are able to refund their coins from these addresses with pre-signed transactions. Our solution works with all relevant blockchains because it needs only standard signatures and a joint public key. Further, only one side needs timed transactions, and if each party acts in its best interest, there are two possible outcomes: If progress stalls, the side with timelocks refunds after a certain time, and the other side refunds immediately. Otherwise, one party redeems the other party’s coins with pre-signed transactions, and the other party completes the swap after a timeout to prevent race conditions. We prove security in the universal composability framework and show privacy preservation via unlinkability and fungibility.

Keywords: cryptocurrencies · atomic swaps · adaptor signatures · timed transactions · UC-framework · privacy · htcls

1 Introduction

Atomic swaps let two mutually distrustful parties exchange different cryptocurrencies across separate blockchains without a trusted third party. When exchanging assets, an atomic swap guarantees exactly one of two outcomes [Her18]: either both transfers execute, or both abort and both parties receive refunds. This atomicity yields an all-or-nothing semantics across ledgers: a single authorization by party *A* that makes party *B*’s coins spendable simultaneously transfers party *A*’s coins to party *B*, preventing unilateral gain and partial execution.

Cross-chain exchanges are not inherent to a blockchain; the participating blockchain(s) must explicitly support them. Due to the importance of exchanging assets, most blockchains support atomic swaps, including Bitcoin [Nak08], Litecoin [Lee11], Ethereum [But13],

E-mail: paul.roesler@fau.de (Paul Rösler), rina.schambach@fau.de (Katharina Schambach), julian.thomas@fau.de (Julian Thomas)

^aCorresponding author



and Solana [Yak17]. These systems implement swaps using hashed timelock contracts (HTLCs), which originate from online forums [Tie13; gma12] and are formalized by Poon and Dryja [PD16]. An HTLC locks the coins of parties A and B under a hashlock and a timelock. If the swap aborts, each party refunds their coins after the time specified by the timelock. Otherwise, when A redeems B 's coins, A reveals the hash preimage, enabling B to redeem A 's coins, ensuring atomicity. However, hashlocks and timelocks constitute complex on-chain functionalities, which makes HTLCs less efficient and enables attackers to hinder the transaction flow by posting many unredeemed hashed timelocks. Moreover, HTLCs do not inherently provide privacy properties: the same hash appears on both blockchains, so observers can connect the two swap transactions [DH20].

Recent research indicates that adaptor signatures [Poe17; Aum+21] are a more efficient alternative to HTLCs. Conceptually, adaptor signatures have three components: a witness, a pre-signature, and a valid signature. The idea of adaptor signatures is that any party holding the witness can complete the pre-signature to form a valid signature; conversely, once that valid signature is published on-chain, the witness is automatically revealed to the counterparty holding the pre-signature. In an atomic swap, this extractability replaces the hashlock: publishing a signature to claim coins on one chain reveals the witness, which the other party can use to claim coins on the other chain. Besides better efficiency, replacing the hash with an (adaptor) signature enhances privacy since the resulting transaction(s) look like any other standard transaction(s). However, some form of timed transactions is still required to preserve fairness. Most existing adaptor signature protocols [Gug20; Abf21; TZY22; Bao+23; Ji+24] require such timelocks on all chains and are limited to 1-to-1 [Gug20; Abf21; Bao+23], 1-to-n [TZY22], or n-to-n [Ji+24] swaps. We do not build upon these, as extending 1-to-1 or n-to-n protocols to the n-to-m case (with $n \neq m$) is not trivial without losing atomicity, while the 1-to-n approach [TZY22] relies on a centralized party.

Thyagarajan et al. [TMM22] present a general n-to-m atomic swap framework in the universal composability (UC) setting [Can01], together with concrete instantiations for blockchains supporting ECDSA or Schnorr signatures. Their approach replaces on-chain timelocks with Verifiable Timed Dlogs (VTDs), i.e., publicly verifiable timelock puzzles that act as an off-chain replacement for timelocks. In the optimized variant, they batch the per-coin VTDs so that each party needs to create (and open) only a single VTD. Nevertheless, VTDs are still required on both sides and impose a significant computational overhead. Hoenisch et al. [Hoe+22] present LightSwap, a 1-to-1 atomic swap protocol for Bitcoin and Monero in which only one party requires timelocks. To achieve this, LightSwap combines (threshold) adaptor signatures with a delay step on the Bitcoin side: Bitcoins are routed through an intermediate timelocked output, creating a bounded window in which the other party can extract the adaptor witness and finish the swap. Although only the Bitcoin side requires timelocks, LightSwap is inherently limited to 1-to-1 swaps. Extending Lightswap to the n-to-n case is non-trivial, since naively running n independent 1-to-1 instances does not enforce atomicity: an adversary can complete only a subset and abort the rest. Further, LightSwap gives no clear direction toward an n-to-m generalization.

Consequently, there exists no generic atomic swap protocol that supports n-to-m swaps across multiple blockchains while requiring timed transactions for only one party's side. In this work, we close this gap by presenting such a protocol, combining and extending the approaches of Thyagarajan et al. [TMM22] and Hoenisch et al. [Hoe+22]. Essentially, our construction builds on the n-to-m framework by Thyagarajan et al. [TMM22], but omits the use of VTDs. Instead, we use standard on-chain timed transactions and, to require them on only one party's chains, we add a delay step similar to that by Hoenisch et al. [Hoe+22]. This leads to the following contributions:

- We present the first atomic swap protocol for n-to-m swaps with timed transactions on only one party's chains.

- We provide an ideal functionality and a formal security proof for our protocol in the universal composability framework [Can01], which ensures correctness when both parties are honest and soundness when one or both parties are dishonest.
- We prove privacy requirements such as unlinkability (of transactions) and fungibility (atomic swap are indistinguishable from normal transactions).

1.1 Technical Overview

We propose Π_{Swap} , an efficient and privacy-preserving alternative to standard Hashed Timelock Contracts (HTLCs). Essentially, our construction combines the insights from Thyagarajan et al. [TMM22] and LightSwap [Hoe+22], which allows us to eliminate the need for timelocks on one party's chains while preserving n-to-m swaps.

HTLCs Hashed Timelock Contracts (HTLCs) enable a 1-to-1 atomic swap between two parties, P_0 and P_1 . The idea is that initially P_0 generates a secret PIN , publishes the hash $h := H(PIN)$, and locks their coin(s) conditioned on h . Subsequently, P_1 locks their coin(s) with the same hash h . Now, P_0 can claim P_1 's coins with PIN , or P_1 can refund after a specified time T^{10} . Similarly, P_1 can claim P_0 's coins with the knowledge of PIN ; otherwise, P_0 can refund after a specified time T^{01} . To execute the swap, P_0 claims P_1 's coins, revealing PIN on-chain and subsequently, P_1 uses the observed PIN to claim P_0 's coins. We note that refunds are available after the timeout expiry, and to prevent race conditions, T^{01} must be strictly greater than T^{10} .

Atomic swaps based on HTLCs inherently suffer from complex on-chain functionalities and a loss of privacy [DH20]. To address these issues, recent approaches [Poe17; Aum+21], including Thyagarajan et al. [TMM22] and Hoenisch et al. [Hoe+22], utilize adaptor signatures as an alternative. We follow this line of work with our protocol. The idea of adaptor signatures is to extend a standard digital signature scheme by binding a signature to a cryptographic statement Y (where Y corresponds to a secret witness y). Conceptually, adaptor signatures work as follows: (1) Pre-Sign: A signer generates a pre-signature $\tilde{\sigma}$ that is invalid but cryptographically tied to Y ; (2) Verify: A verifier checks that $\tilde{\sigma}$ is correct relative to Y ; (3) Adapt: A party holding y can transform $\tilde{\sigma}$ into a valid signature σ ; and (4) Extract: Any party holding $\tilde{\sigma}$ who observes σ on-chain can compute the witness y .

SP'22 Thyagarajan, Malavolta, and Moreno-Sanchez [TMM22] propose a generic framework for n-to-m atomic swaps. Their concrete construction assumes ECDSA or Schnorr signatures and utilizes Verifiable Timed Dlogs (VTDs) instead of standard timelocks. The idea of their protocol is that initially, P_0 locks their coins into a joint address pk^{01} , which allows P_0 to refund via a VTD after time T^0 . Similarly, P_1 locks their coins with pk^{10} , refundable via a VTD after T^1 (where $T^1 < T^0$). Once locked, P_0 generates a witness pair $(y^{\text{transfer}}, Y^{\text{transfer}})$ and the parties exchange adaptor signatures for the swap transactions. To execute, P_0 claims P_1 's coins using the witness y^{transfer} via $\sigma^{10} \leftarrow \text{Adapt}(\tilde{\sigma}^{10}, y^{\text{transfer}})$. Finally, P_1 extracts y^{transfer} from the on-chain signature σ^{10} and uses it to complete the swap by claiming P_0 's coins via σ^{01} .

The primary shortcoming of Thyagarajan et al. [TMM22] is that their protocol inherently requires VTDs for all parties to provide timelock-like functionality. We avoid VTDs entirely and require timed transactions on only one party's blockchains. Concretely, on P_1 's (possibly multiple) blockchains, coins are locked with standard on-chain timed transactions, while on P_0 's blockchains, coins are locked in joint signature addresses. To preserve fairness, we follow a similar idea as LightSwap [Hoe+22]: a delay step ensures that redeeming on P_1 's side reveals the adaptor witness, while a delay address and a standard timelock enforce a time window in which P_1 can extract the witness and redeem P_0 's coins before any refund becomes executable, thus preventing race conditions.

Table 1: Comparison of our protocol Π_{Swap} with related atomic swap approaches. Here, costs refer to the total number of on-chain transactions for a successful execution.

	HTLCs	SP'22 [TMM22]	LightSwap [Hoe+22]	Π_{Swap} (Ours)
Primitive	Hashlocks, Timelocks	Adaptor signatures, Verifiable Timed Dlogs	Threshold adaptor signatures	Key-aggr. two-party adaptor signatures
Timelocks	All chains	All parties	One chain	One party's chains
Assets	1-to-1	n-to-m	1-to-1	n-to-m
Costs	4	$2n + 2m$	5	$2n + 3m$
Privacy	Linkable	Unlinkable, Fungible	Unlinkable, Fungible	Unlinkable, Fungible

LightSwap. Hoenisch, Mazumdar, Moreno-Sanchez, and Ruj [Hoe+22] present a 1-to-1 atomic swap protocol for Bitcoin and Monero, where only the Bitcoin side uses timed transactions. The idea of LightSwap is as follows: First, the parties (P_0 on the Monero side, P_1 on the Bitcoin side) lock their coin(s) into joint signature outputs pk^{01} and pk^{10} . On the Monero side, P_0 additionally prepares a refund transaction in advance, and the parties jointly pre-sign it, so that P_0 can reclaim the coin(s) in case of an abort. Next, P_1 provides P_0 with a pre-signature for the Bitcoin redeem transaction that spends from pk^{10} into a delay output pk^{delay} . To execute the swap, P_0 adapts P_1 's Bitcoin pre-signature with P_0 's witness and publishes the resulting redeem transaction. Then, P_1 extracts the witness and uses it to redeem the Monero coins from pk^{01} . To prevent race conditions while using timed transactions only on the Bitcoin side, the Bitcoin coins are routed through the delay output pk^{delay} and the final transfer is only possible after time T^{transfer} .

LightSwap is limited to 1-to-1 swaps between Bitcoin and Monero. Generalizing LightSwap to the n-to-m case is not straightforward, because atomicity is not guaranteed. For example, in the n-to-n case (running LightSwap n times), no functionality ensures that either all n swaps are done or none. Beyond that, LightSwap does not provide any functionality for the n-to-m case (with $n \neq m$). We address this by combining LightSwap's delay step idea with the n-to-m structure of Thyagarajan et al. [TMM22]: we use a single witness pair $(y^{\text{transfer}}, Y^{\text{transfer}})$ and create adaptor signatures $\tilde{\sigma}^{01}$ and $\tilde{\sigma}^{10}$ for all swap transactions under Y^{transfer} , so extracting y^{transfer} suffices to redeem all coins. Moreover, rather than relying on chain-specific constructions, we adopt the generic framework by [TMM22] and define our protocol via the generic interface of (two-party-)adaptor signatures.

Further, we identify and address minor shortcomings of LightSwap: 1. In LightSwap, a malicious party B can lock the counterparty's coins without providing a valid refund path. We fix this by requiring both parties to first generate an adaptor signature $\tilde{\sigma}^{\text{refund}}$ for the refund transaction before any signatures are revealed. 2. LightSwap uses the counterparty's signature in the lock transaction, although a party should be able to lock coins using only its own secret key; we remove this dependency. 3. On the Bitcoin side, coins are moved from an output $pk_A + pk_B$ back into the same address, so both outputs share the same secret key. We replace this with distinct addresses pk^{10} and pk^{delay} to ensure coins are actually transferred. 4. In LightSwap, an emergency refund is created by adapting B 's own precomputed signature, which allows unilateral execution. We instead require a jointly created adaptor signature $\tilde{\sigma}^{\text{refund}}$, such that the refund signature σ^{refund} can only be derived by extracting the witness y^{refund} from the counterparty's refund transaction.

Our Protocol Π_{Swap} . We present an n-to-m atomic swap protocol using joint signatures on all blockchains, while requiring standard timelocks only on P_1 's chains. The high level idea of our protocol is as follows: Before revealing any signature, the parties create

Table 2: Comparison of on-chain transaction costs across atomic swap protocols.

Protocol	Case	Init	Locking	Delay	Redeem	Total
Our protocol	n-to-m	0	$n + m$	m	$n + m$	$2n + 3m$
[TMM22]	n-to-m	0	$n + m$	0	$n + m$	$2n + 2m$
[Hoe+22]	1-to-1	0	2	1	2	5
HTLCs	1-to-1	0	2	0	2	4

adaptor signatures for P_0 's refund transactions. Then, P_0 locks their coins into joint addresses pk^{01} . Similarly, P_1 locks their coins into joint addresses pk^{10} and establishes standard timelock refunds after time T^{refund} . Once both sides are locked, P_0 samples a witness pair $(y^{\text{transfer}}, Y^{\text{transfer}})$ and the parties create adaptor signatures for the swap transactions under Y^{transfer} . Crucially, we rely on pre-verify soundness [Ger+24], which allows P_1 to verify that these pre-signatures are valid and extractable with respect to the statement Y^{transfer} without needing knowledge of the witness y^{transfer} , thus ensuring P_1 can redeem P_0 's coins when P_0 redeems P_1 's coins. To execute, P_0 redeems P_1 's coins by adapting the corresponding pre-signatures, which reveals y^{transfer} to P_1 . Finally, P_1 extracts y^{transfer} from the on-chain signature and uses it to redeem P_0 's coins. To prevent race conditions, P_1 's coins are first moved into joint delay addresses pk^{delay} , from which P_0 can transfer to their final addresses only via a standard timelocked transaction that becomes valid after T^{transfer} . If the swap aborts, P_0 refunds at any time, and P_1 refunds either after T^{refund} or, if coins were already moved to the delay address pk^{delay} , by spending them back to P_1 's refund address using the pre-signed refund transaction from pk^{delay} .

Cost Comparison On a high level, our protocol has communication costs linear in the total number of swapped coins, that is, $\mathcal{O}(n+m)$, similarly to the protocol by Thyagarajan, Malavolta, and Moreno-Sanchez [TMM22], whereas the protocol by Hoenisch, Mazumdar, Moreno-Sanchez, and Ruj [Hoe+22] trivially has constant communication costs since it only covers the 1-to-1 case. For a more concrete comparison, we refer to Table 2, which summarizes the number of on-chain transactions. We categorize the protocol execution in four phases: Initialization (Init), Locking, Delay, and Redeem, while assuming a successful n-to-m (or 1-to-1) atomic swap, where keys already exist, and parameters are agreed on as well as exchanged. As a result, we observe that the number of on-chain transactions by [Hoe+22] and our protocol in the $n = m = 1$ case are equivalent. Moreover, we see that [TMM22] only requires $2n + 2m$ on-chain transactions, whereas our protocol requires $2n + 3m$ on-chain transactions. The reason that our protocol requires m more on-chain transactions than the protocol by [TMM22] is that, inspired by [Hoe+22], our protocol has an additional delay step.

1.2 Further Related Work

TierNolan [Tie13] first introduces atomic swaps on the BitcoinTalk forum, and the Decred Project [Yoc17] later demonstrates practical feasibility through the first on-chain swap between Decred and Litecoin. Subsequently, Herlihy [Her18] formalizes atomic swaps in distributed systems, and Thomas and Schwartz [TS15] generalize these protocols to broader payment networks. Most existing solutions for atomic swaps utilize hashed timelock contracts (HTLCs) [AK25; Cla+24; Her18; Li+24; Maz22; Wad+22; XAD21]. However, there are several shortcomings with HTLC-based approaches, leading to adaptor signatures being a promising alternative. Adaptor signatures originate from the work of Poelstra [Poe17]. Aumayr et al. [Aum+21] formalize adaptor signatures by describing adaptability and extractability in detail. Dai et al. [DOY22] extend this line of work by

introducing full extractability. Gerhart et al. [Ger+24] further fill gaps in previous security definitions by introducing pre-verify soundness, which ensures that faulty statements cannot produce a verifiable pre-signature. They also provide secure constructions that fulfill these requirements. Notably, Ji et al. [Ji+24] define multi-adaptor signatures, where multiple parties obtain parts of the secret key and have to cooperate to create a valid adaptor signature. Finally, concurrent work by Gerhart et al. [GRS25] provides a UC functionality for adaptor signatures. We view this as complementary related work and, in line with prior atomic-swap works [TMM22; Hoe+22], consider a game-based treatment sufficient for our purposes here. A fully UC-based treatment of the adaptor-signature component in atomic swaps is therefore left for future work. Most existing adaptor signature protocols [Gug20; Abf21; TZY22; Bao+23; Ji+24] require timelocks for all parties and chains. Further, these are limited to 1-to-1 [Gug20; Abf21; Bao+23], 1-to-n [TZY22], or n-to-n [Ji+24] swaps. We do not build upon these, as extending 1-to-1 or n-to-n protocols to the n-to-m case is not trivial without losing atomicity, while the 1-to-n approach [TZY22] relies on a centralized party. Thyagarajan et al. [TMM22] provide a generalized framework for n-to-m swaps with a concrete construction for any two blockchains supporting ECDSA [Ame05] or Schnorr [Sch90] signatures, which they utilize to implement the necessary adaptor signature functionalities. While their protocol does not strictly require on-chain timelocks, it requires both parties to support Verified Timed Discrete Logs, an off-chain substitute with significant computational overhead. Conversely, Hoenisch et al. [Hoe+22] introduce a protocol for 1-to-1 swaps for Bitcoin and Monero with timed transactions only on the Bitcoin side. As a result, all of these solutions, except for Hoenisch et al. [Hoe+22], require both sides to support (on-chain or off-chain) functionalities for timelocks. Currently, there exists no generic solution for n-to-m swaps where only one party requires timed transactions.

2 Preliminaries

Notations. The security parameter is $\lambda \in \mathbb{N}$. For algorithm inputs, we use 1^λ to ensure that the runtime of the algorithm is polynomial in λ . Furthermore, we denote negligible functions as $\text{negl}(\lambda)$. A function f is negligible if for every $k \in \mathbb{N}$ there exists $N \in \mathbb{N}$ such that for all integers $n \geq N$ it holds that $|f(n)| \leq n^{-k}$. For notation, we use $\leftarrow$ for a (possibly) probabilistic output of an algorithm. In contrast, $=$ and $:=$ are used for a deterministic assignment. For transactions, keys, and signatures, the indices at the bottom represent the associated party, while the indices at the top indicate their functionality. Generally, 01 is associated with the transaction from P_0 to P_1 while 10 refers to the reverse transaction from P_1 to P_0 .

Signatures. A digital signature scheme verifies that a message is authentic, unmodified, and linked to the sender’s identity.

Definition 1 (Signatures). A (digital) signature scheme Π_{DS} is composed of three (probabilistic) polynomial-time (PPT) algorithms:

$(pk, sk) \leftarrow \text{KGen}(1^\lambda)$: The key generation algorithm takes as input a security parameter λ and outputs a pair of keys (pk, sk) , with public key pk and a secret key sk .

$\sigma \leftarrow \text{Sign}(sk, m)$: The signing algorithm takes as input a secret key sk and a message m , and returns a signature σ .

$0/1 \leftarrow \text{Vrfy}(pk, m, \sigma)$: The verification algorithm takes as input a public key pk , a message m , and a signature σ . It outputs 1, if the message/signature-pair is valid for pk , otherwise it outputs 0.

EUFCMA $_{\mathcal{A}, \Pi_{\text{DS}}}(\lambda)$	EUFCMA $_{\mathcal{A}, \Pi_{\text{MuS}}}(\lambda)$
1 : $\mathcal{Q} := \emptyset$	1 : $\mathcal{Q} := \emptyset$
2 : $(pk, sk) \leftarrow \text{KGen}(1^\lambda)$	2 : $(pk_2, sk_2) \leftarrow \text{KGen}(1^\lambda)$
3 : $(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{DS}}}(pk)$	3 : $(pk_1, m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuS}}}(pk_2)$
4 : $b^* = (m^* \notin \mathcal{Q}) \wedge \text{Vrfy}(pk, m^*, \sigma^*)$	4 : $pk \leftarrow \text{KAgg}(pk_1, pk_2)$
5 : return b^*	5 : $b^* = (m^* \notin \mathcal{Q}) \wedge \text{MuVrfy}(pk, m^*, \sigma^*)$
$\mathcal{O}_{\text{DS}}(m)$	$\mathcal{O}_{\text{MuS}}(m)$
6 : $\sigma \leftarrow \text{Sign}(sk, m)$	7 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$
7 : $\mathcal{Q} := \mathcal{Q} \cup \{m\}$	8 : $\sigma_2 \leftarrow \text{Sign}(sk_2, m)$
8 : return σ	9 : return σ_2

Figure 1: Signature unforgeability game EUFCMA $_{\mathcal{A}, \Pi_{\text{DS}}}$ and two-party unforgeability game EUFCMA $_{\mathcal{A}, \Pi_{\text{MuS}}}$.

For correctness, we require: For any security parameter λ , any $(pk, sk) \leftarrow \text{KGen}(1^\lambda)$, and any message m , it holds that $\text{Vrfy}(pk, m, \text{Sign}(sk, m)) = 1$ except with negligible probability.

For security, existential unforgeability under an adaptive chosen message attack (EUFCMA) guarantees that a PPT adversary $\mathcal{A}$ who has access to a signing oracle cannot forge a valid signature on a message that $\mathcal{A}$ has not queried to the oracle (except with negligible probability).

Definition 2 (EUFCMA-security). We require that for every PPT adversary $\mathcal{A}$

$$\Pr[\text{EUFCMA}_{\mathcal{A}, \Pi_{\text{DS}}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game EUFCMA $_{\mathcal{A}, \Pi_{\text{DS}}}$ is defined in Figure 1.

Two-party Signatures. In our scheme, we further require that two parties co-sign a message, meaning that each party owns a secret key, but only if both keys are combined, the signature becomes valid. Therefore, we introduce so-called two-party signature schemes, which we state as a simplification of multi-signatures [Bol03].

Definition 3 (Two-party signatures). A two-party signature scheme Π_{MuS} with parties P_1, P_2 is composed of three PPT algorithms:

$(pk, sk_1, sk_2) \leftarrow \text{MuKGen}(1^\lambda)$: The interactive key generation algorithm takes as input a security parameter λ and outputs a pair of keys to each party P_i , consisting of a common public key pk and a distinct secret key sk_i .

For the security game (see Definition 4) and the protocol description (see Figure 7), we subdivide this function: Each party runs the normal key generation algorithm $(pk_1, sk_1) \leftarrow \text{KGen}(1^\lambda)$ resp. $(pk_2, sk_2) \leftarrow \text{KGen}(1^\lambda)$, and then they both obtain the common public key with a local key aggregation algorithm $pk \leftarrow \text{KAgg}(pk_1, pk_2)$.

$\sigma \leftarrow \text{MuSign}(sk_1, sk_2, m)$: The interactive two-party signature generation algorithm takes as input the secret keys sk_i of both parties and a message m , and returns a joint signature σ .

For the security game (see Definition 4) and the protocol description (see Figure 7),

we subdivide this function: Each party runs the normal signing algorithm $\sigma_1 \leftarrow \text{Sign}(sk_1, m)$ resp. $\sigma_2 \leftarrow \text{Sign}(sk_2, m)$, and then they both obtain the joint signature with a local signature aggregation algorithm $\sigma \leftarrow \text{SigAgg}(\sigma_1, \sigma_2)$.

$0/1 \leftarrow \text{MuVrfy}(pk, m, \sigma)$: The non-interactive verification algorithm takes as input the public key pk , a message m and a signature σ . It outputs 1, if the message/signature-pair is valid for pk , otherwise it outputs 0.

For correctness, we require: For any security parameter λ , any $(pk, sk_1, sk_2) \leftarrow \text{MuKGen}(1^\lambda)$, and any message m , it holds that $\text{MuVrfy}(pk, m, \text{MuSign}(sk_1, sk_2, m)) = 1$ except with negligible probability.

For security, we require existential unforgeability under an adaptive chosen message attack (EUF-CMA), which guarantees unforgeability for messages for which the adversary has not seen a valid signature before [BPW23]. Moreover, one of the two parties can be corrupted by the adversary $\mathcal{A}$, who can therefore pick the secret key of that party; for the other party, $\mathcal{A}$ is provided with a signature oracle.

Definition 4 (EUF-CMA-security). We require that for every PPT adversary $\mathcal{A}$

$$\Pr[\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{Mus}}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game $\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{Mus}}}$ is defined in Figure 1 (w.l.o.g. we assume that $\mathcal{A}$ corrupts P_1).

Remark 1. This definition is kept as broad as possible in order to include most protocols for multi-signatures, threshold signatures (with $m = n$), and aggregate signatures in the literature [Bol03; BPW23; Kar+24].

Adaptor Signatures. Adaptor signatures [Aum+21] generate a variant of regular signatures by embedding a secret witness into a so-called pre-signature. The person who knows this secret witness can use it adapt the pre-signature into a regular signature, which in turn enables the other person to extract the witness from the pre-signature, when the regular signature is revealed.

Definition 5 (Hard relations). A relation R is defined as a set of pairs (y, Y) , where y is a witness and Y is a corresponding statement. The pair (y, Y) belongs to R if and only if y is a valid witness for Y . The relation R is said to be *hard* if, given only a statement Y , the probability of calculating a witness y such that $(y, Y) \in R$ is negligible. Determining whether $(y, Y) \in R$ and sampling valid pairs from R should be computationally efficient. We use this generally for the relation between public and secret keys in signatures (Definition 1) and particularly for the construction of adaptor signatures (Definition 6).

Definition 6 (Adaptor Signatures). An adaptor signature scheme Π_{AS} builds on an underlying signature scheme $\Pi_{\text{DS}} = (\text{KGen}, \text{Sign}, \text{Vrfy})$ and is composed of four PPT algorithms:

$\tilde{\sigma} \leftarrow \text{pSign}(sk, m, Y)$: The pre-signing algorithm takes as input a secret key sk , a message m and a statement Y , which fulfills the hard relation $(y, Y) \in R$, and returns a pre-signature $\tilde{\sigma}$.

$0/1 \leftarrow \text{pVrfy}(pk, m, Y, \tilde{\sigma})$: The pre-verifying algorithm takes as input a public key pk , a message m , a statement Y , and a pre-signature $\tilde{\sigma}$. It outputs 1, if the message/pre-signature-pair is valid for pk and Y , otherwise it outputs 0.

$y \leftarrow \text{Ext}(\sigma, \tilde{\sigma}, Y)$: The extraction algorithm takes as input a signature σ , a pre-signature $\tilde{\sigma}$ and a statement Y , and returns the corresponding witness y that fulfills $(y, Y) \in R$. If this is not possible, i.e. the witness extracted from σ and $\tilde{\sigma}$ does not match the statement, it returns $\perp$.

$\sigma \leftarrow \text{Adapt}(\tilde{\sigma}, y)$: The adaptation algorithm takes as input a pre-signature $\tilde{\sigma}$ and a witness y , and outputs the signature σ .

We do not elaborate the security requirements for adaptor signatures here, as we do not use normal adaptor signatures in our scheme but instead so-called two-party adaptor signatures.

Two-party Adaptor Signatures. Following the two-party signatures in Definition 3, which we use as a fundamental building block in our scheme to ensure fairness for the swap, we require corresponding two-party adaptor signatures. Again, both parties own part of the secret key and only together can they create a valid adaptor signature [Erw+21].

Definition 7 (Two-party adaptor signatures). A two-party adaptor signature scheme Π_{MuAS} with parties P_1, P_2 builds on an underlying two-party signature scheme $\Pi_{\text{MuS}} = (\text{MuKGen}, \text{MuSign}, \text{MuVrfy})$ and is composed of four PPT algorithms:

$\tilde{\sigma} \leftarrow \text{MuPSign}(sk_1, sk_2, m, Y)$: The interactive two-party pre-signature generation algorithm takes as input the secret keys sk_i of both parties P_i , a message m and a statement Y , which fulfills the hard relation $(y, Y) \in R$, and returns a joint pre-signature $\tilde{\sigma}$. For the security games (see Definitions 9 and 11) and the protocol description (see Figure 7), we subdivide this function: Each party runs the pre-signing algorithm $\tilde{\sigma}_1 \leftarrow \text{pSign}(sk_1, m, Y)$ resp. $\tilde{\sigma}_2 \leftarrow \text{pSign}(sk_2, m, Y)$, and then both obtain the joint pre-signature with a local pre-signature aggregation algorithm $\tilde{\sigma} \leftarrow \text{pSigAgg}(\tilde{\sigma}_1, \tilde{\sigma}_2)$.

$0/1 \leftarrow \text{MuPVrfy}(pk, m, Y, \tilde{\sigma})$: The non-interactive pre-verification algorithm takes as input an aggregate public key pk , a message m , a statement Y , and a pre-signature $\tilde{\sigma}$. It outputs 1, if the message/pre-signature-pair is valid for pk and Y , and otherwise 0.

$y \leftarrow \text{MuExt}(\sigma, \tilde{\sigma}, Y)$: The non-interactive extraction algorithm takes as input a signature σ , a pre-signature $\tilde{\sigma}$ and a statement Y , and returns the corresponding witness y that fulfills $(y, Y) \in R$. If this is not possible, i.e. the witness extracted from σ and $\tilde{\sigma}$ does not match the statement, it returns $\perp$.

$\sigma \leftarrow \text{MuAdapt}(\tilde{\sigma}, y)$: The non-interactive adaptation algorithm takes as input a pre-signature $\tilde{\sigma}$ and a witness y , and outputs the signature σ .

The two-party adaptor signatures must fulfill correctness and existential unforgeability under an adaptive chosen message attack (aEUF-CMA). Beyond, we require witness extractability and pre-signature adaptability according to the definitions used by Ji et al. [Ji+24]. Furthermore, we extend the new notions of unlinkability [DOY22] and computational pre-verify soundness [Ger+24] for two-party adaptor signatures. In doing so, we use MuKGen, KAgg, MuSign, and MuVrfy as the two-party signature algorithms from Definition 3.

Correctness. First, we require correctness: If both parties follow the protocol honestly, the pre-signature must be verifiable, adaptable with the witness into a valid signature, and the witness must be extractable with the adapted signature.

Definition 8 (Correctness). For any security parameter λ , any $(pk, sk_1, sk_2) \leftarrow \text{MuKGen}(1^\lambda)$, any $(y, Y) \in R$, and any message m , it holds for $\tilde{\sigma} \leftarrow \text{MuPSign}(sk_1, sk_2, m, Y)$ that

$$\Pr[\text{MuPVrfy}(pk, m, Y, \tilde{\sigma}) = 1 \wedge \text{MuVrfy}(pk, m, \text{MuAdapt}(\tilde{\sigma}, y)) = 1 \\ \wedge (\text{MuExt}(\text{MuAdapt}(\tilde{\sigma}, y), \tilde{\sigma}, Y), Y) \in R] \geq 1 - \text{negl}(\lambda),$$

where negl is a negligible function.

$a\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda)$	$a\text{WitExt}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda)$	$\text{Unlink}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda)$
$1: \mathcal{Q} := \emptyset$ $2: (pk_2, sk_2) \leftarrow \text{KGen}(1^\lambda)$ $3: (y, Y) \in R$ $4: (pk_1, m^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuS}}, \mathcal{O}_{\text{MuAS}}}(pk_2, Y)$ $5: \tilde{\sigma}_2^* \leftarrow \text{pSign}(sk_2, m^*, Y)$ $6: \sigma^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuS}}, \mathcal{O}_{\text{MuAS}}}(\tilde{\sigma}_2^*)$ $7: pk \leftarrow \text{KAgg}(pk_1, pk_2)$ $8: b^* = (m^* \notin \mathcal{Q})$ $9: \quad \wedge \text{MuVrfy}(pk, m^*, \sigma^*)$ $10: \text{ return } b^*$	$1: \mathcal{Q} := \emptyset$ $2: (pk_2, sk_2) \leftarrow \text{KGen}(1^\lambda)$ $3: (pk_1, m^*, Y^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuS}}, \mathcal{O}_{\text{MuAS}}}(pk_2)$ $4: \tilde{\sigma}_2^* \leftarrow \text{pSign}(sk_2, m^*, Y^*)$ $5: (\sigma^*, \tilde{\sigma}_1^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuS}}, \mathcal{O}_{\text{MuAS}}}(\tilde{\sigma}_2^*)$ $6: \tilde{\sigma}^* \leftarrow \text{pSigAgg}(\tilde{\sigma}_1^*, \tilde{\sigma}_2^*)$ $7: y^* \leftarrow \text{MuExt}(\sigma^*, \tilde{\sigma}^*, Y^*)$ $8: pk \leftarrow \text{KAgg}(pk_1, pk_2)$ $9: b^* = (m^* \notin \mathcal{Q}) \wedge ((y^*, Y^*) \notin R)$ $10: \quad \wedge \text{MuVrfy}(pk, m^*, \sigma^*)$ $10: \text{ return } b^*$	$1: b \leftarrow \$\{0, 1\}$ $2: (pk, sk_1, sk_2) \leftarrow \text{MuKGen}(1^\lambda)$ $3: (m^*, y^*, Y^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuSign}}, \mathcal{O}_{\text{MuPSign}}}(pk)$ $4: \text{Assert } (y^*, Y^*) \in R$ $5: \tilde{\sigma}^* \leftarrow \text{MuPSign}(sk_1, sk_2, m^*, Y^*)$ $6: \sigma_1 \leftarrow \text{MuAdapt}(\tilde{\sigma}^*, y^*)$ $7: \sigma_0 \leftarrow \text{MuSign}(sk_1, sk_2, m^*)$ $8: b^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{MuSign}}, \mathcal{O}_{\text{MuPSign}}}(\sigma_b)$ $9: \text{ return } (b = b^*)$
$\mathcal{O}_{\text{MuS}}(m)$ $11: \mathcal{Q} := \mathcal{Q} \cup \{m\}$ $12: \sigma_2 \leftarrow \text{Sign}(sk_2, m)$ $13: \text{ return } \sigma_2$	$\mathcal{O}_{\text{MuS}}(m)$ $11: \mathcal{Q} := \mathcal{Q} \cup \{m\}$ $12: \sigma_2 \leftarrow \text{Sign}(sk_2, m)$ $13: \text{ return } \sigma_2$	$\mathcal{O}_{\text{MuSign}}(m)$ $10: \sigma \leftarrow \text{MuSign}(sk_1, sk_2, m)$ $11: \text{ return } \sigma$
$\mathcal{O}_{\text{MuAS}}(m, Y)$ $14: \mathcal{Q} := \mathcal{Q} \cup \{m\}$ $15: \tilde{\sigma}_2 \leftarrow \text{pSign}(sk_2, m, Y)$ $16: \text{ return } \tilde{\sigma}_2$	$\mathcal{O}_{\text{MuAS}}(m, Y)$ $14: \mathcal{Q} := \mathcal{Q} \cup \{m\}$ $15: \tilde{\sigma}_2 \leftarrow \text{pSign}(sk_2, m, Y)$ $16: \text{ return } \tilde{\sigma}_2$	$\mathcal{O}_{\text{MuPSign}}(m, Y)$ $12: \tilde{\sigma} \leftarrow \text{MuPSign}(sk_1, sk_2, m, Y)$ $13: \text{ return } \tilde{\sigma}$

Figure 2: Overview for the two-party adaptor unforgeability game $a\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{MuAS}}}$, witness extractability game $a\text{WitExt}_{\mathcal{A}, \Pi_{\text{MuAS}}}$, and unlinkability game $\text{Unlink}_{\mathcal{A}, \Pi_{\text{MuAS}}}$.

aEUF-CMA-security. For security, aEUF-CMA-security guarantees that a PPT adversary $\mathcal{A}$ who corrupts one party and picks the secret key of that party cannot forge a valid signature on a message that $\mathcal{A}$ has not queried to the signature or the pre-signature oracle (except with negligible probability). The adversary is only provided with one pre-signature on the challenge message.

Definition 9 (aEUF-CMA-security). We require that for every PPT adversary $\mathcal{A}$

$$\Pr[a\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game $a\text{EUF-CMA}_{\mathcal{A}, \Pi_{\text{MuAS}}}$ is defined in Figure 2 (w.l.o.g. we assume that $\mathcal{A}$ corrupts P_1).

Adaptability. Adaptability guarantees that any valid pre-signature can be adapted into a valid signature with the corresponding witness.

Definition 10 (Adaptability). For any public key pk , any $(y, Y) \in R$, any message m , and any joint adaptor signature $\tilde{\sigma}$ that fulfill $\text{MuPVrfy}(pk, m, Y, \tilde{\sigma}) = 1$ it holds that

$$\Pr[\text{MuVrfy}(pk, m, \text{MuAdapt}(\tilde{\sigma}, y)) = 1] \geq 1 - \text{negl}(\lambda),$$

where negl is a negligible function.

Extractability. Extractability guarantees that it is always possible to extract the witness from a pre-signature and a valid signature on the same message if no other valid (pre-)signatures are known on that message. In this case, one of the parties is corrupted by the adversary $\mathcal{A}$, who can therefore pick the witness and the secret key of that party and is provided with a signature and a pre-signature oracle.

Definition 11 (Extractability). We require that for every PPT adversary $\mathcal{A}$

$$\Pr[a\text{WitExt}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda) = 1] \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game $a\text{WitExt}_{\mathcal{A}, \Pi_{\text{MuAS}}}$ is defined in Figure 2 (w.l.o.g. we assume that $\mathcal{A}$ corrupts P_1).

Unlinkability. Unlinkability guarantees that a joint signature generated by MuSign is computationally indistinguishable from a joint signature generated by MuAdapt and MuPSign, even if the adversary has access to a two-party signature oracle and a two-party adaptor signature oracle.

Definition 12 (Unlinkability). We require that for every PPT adversary $\mathcal{A}$

$$\Pr[\text{Unlink}_{\mathcal{A}, \Pi_{\text{MuAS}}}(\lambda) = 1] - \frac{1}{2} \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game $\text{Unlink}_{\mathcal{A}, \Pi_{\text{MuAS}}}$ is defined in Figure 2.

Pre-verify soundness. Pre-verify soundness guarantees that a statement Y which is not part of the hard relation R cannot produce a verifiable pre-signature.

Definition 13 (Pre-verify soundness). For $(\cdot, Y) \notin R$, any security parameter λ , $(pk, sk_1, sk_2) \leftarrow \text{MuKGen}(1^\lambda)$, any message m , and any pre-signature $\tilde{\sigma}$ it holds that

$$\Pr[\text{MuPVrfy}(pk, m, Y, \tilde{\sigma}) = 1] \leq \text{negl}(\lambda),$$

where negl is a negligible function.

Remark 2. Dai et al. [DOY22] have replaced both aEUF-CMA-security and extractability with the stronger notion of full extractability where the adversary is allowed multiple pre-signatures on the challenge message m^* instead of just one. This provides a more complete definition of adaptor signatures. However, our scheme does not need this stronger security, as both parties have to cooperate to create a pre-signature ($\tilde{\sigma} \leftarrow \text{MuPSign}(sk_1, sk_2, m, Y)$ requires both parties' secret keys), and the honest party has no incentive to do this more than once on the same message. Additionally, the messages to be signed are unique blockchain transactions [Ger+24], so a malicious party could not acquire another pre-signature for the same message from any other context.

For our scheme, we need a two-party adaptor signature construction that satisfies these requirements for each of the blockchains based on their respective signature schemes, as the blockchains themselves do not usually provide this functionality. Indeed, a generic implementation for identity-based signature schemes exists in the literature [Zhu+23; Erw+21], and multi-party schemes can also be used in the two-party setting, e.g. Ji et al. [Ji+24] present implementations for Schnorr [Sch90] and Dilithium [Duc+18] signatures.

Blockchain Transactions. A transaction on a blockchain moves coins from one address to another (both on the same chain) if proper authentication is provided. We base our definition on the model of Thyagarajan et al. [TMM22].

Definition 14 (Transactions). A blockchain transaction tx consists of three components: 1. pk^{before} : The public key address that held the coins before the swap. 2. pk^{after} : The public key address the coins are transferred to. 3. v : The coins themselves. To authorize such a transaction, a valid signature σ using the secret key sk^{before} corresponding to the original public key address is required. The message that is signed with this signature is the transaction itself. Thus, $tx = (pk^{\text{before}}, pk^{\text{after}}, v)$ is authorized by $\sigma \leftarrow \text{Sign}(sk^{\text{before}}, tx)$.

We extend the definition of transactions to include timers, where a time can be set after which the transaction becomes valid. There are different ways of implementing timers on blockchains, with Ladleif and Weske [LW20] providing a very comprehensive overview

of the options and their advantages and disadvantages. In the description of our protocol, we model the timers with parameters (i.e. additional arguments of the transactions), as they present the easiest implementation for both absolute and relative timers.

Definition 15 (Timed transactions). A timed transaction $tx = (pk^{\text{before}}, pk^{\text{after}}, v, T)$ receives an additional parameter T describing an execution time at which tx becomes valid. In this, T can represent either an absolute point in time or a delay relative to another transaction. For our purposes, we use absolute time parameters T rather than relative ones, because we want multiple transactions that are defined consecutively to all become valid at the same time. Analogous to regular transactions, timed transactions are authorized by signing them with sk^{before} .

Atomic Swaps. An atomic swap denotes the exchange of an agreed-upon amount of cryptocurrencies between two parties without any third-party involvement. For their security, we require correctness and soundness according to the definitions by Hoenisch et al. [Hoe+22]. Additionally, we require unlinkability and fungibility for privacy preservation.

Definition 16 (Security of Atomic Swaps). Let Π be an atomic swap protocol between two parties P_0 and P_1 . For $j \in \{0, 1\}$, let V_j denote the set of assets initially owned by P_j (across one or more blockchains), and let V'_j denote the set of assets owned by P_j upon termination of the protocol.

Correctness: For any execution of Π in which both parties follow the protocol honestly, the protocol terminates with each participant receiving the agreed-upon amount

$$\Pr [V_1 = V'_0 \wedge V_0 = V'_1] \geq 1 - \text{negl}(\lambda),$$

where negl is a negligible function.

Soundness: For any execution of Π in which one party deviates from the protocol, the honest party P_j with $j \in \{0, 1\}$ does not lose funds

$$\Pr [V_j \subseteq V'_j \vee V_{1-j} \subseteq V'_j] \geq 1 - \text{negl}(\lambda),$$

where negl is a negligible function.

Regarding the privacy of atomic swaps, we consider two properties. Again, let Π be an atomic swap protocol, and let $\mathcal{A}$ be any external PPT adversary observing the blockchain state. Then, for *unlinkability*, $\mathcal{A}$ must not be able to link the cross-chain transactions involved in Π , except with negligible probability (see Definition 17). The other property is *fungibility*, where $\mathcal{A}$ must be unable to tell whether any transaction tx appearing on a blockchain (that supports executions of Π) is part of an atomic swap or a standard transaction (see Definition 18).

Definition 17 (Unlinkability). We require that for every PPT adversary $\mathcal{A}$

$$\left| \Pr[\text{Unlink}_{\mathcal{A}, \Pi_{\text{Swap}}}(\lambda) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where negl is a negligible function and the game $\text{Unlink}_{\mathcal{A}, \Pi_{\text{Swap}}}$ is defined in Figure 3.

Definition 18 (Fungibility). Assume the transaction type τ captures all publicly observable structural features of a transaction, and let $\mathcal{T}$ be the set of all transaction types τ that can occur both in honest executions of Π_{Swap} and in honest non-swap transactions on the underlying blockchain. Then, we require that for every PPT adversary $\mathcal{A}$

$$\left| \Pr[\text{Fung}_{\mathcal{A}, \Pi_{\text{Swap}}}(\lambda) = 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda),$$

where negl is a negligible function and the game $\text{Fung}_{\mathcal{A}, \Pi_{\text{Swap}}}(\lambda)$ is defined in Figure 4.

$\text{Unlink}_{\mathcal{A}, \Pi_{\text{Swap}}}(\lambda)$
1 : $b \leftarrow_{\$} \{0, 1\}$ 2 : Choose $(T^{\text{refund},*}, T^{\text{transfer},*})$ 3 : $(TX_A^1, TX_B^1) \leftarrow$ Run honest execution of Π_{Swap} with $(T^{\text{refund},*}, T^{\text{transfer},*})$ // get public transcript (transactions and signatures) on chains A and B 4 : $(TX_A^2, TX_B^2) \leftarrow$ Run honest execution of Π_{Swap} with $(T^{\text{refund},*}, T^{\text{transfer},*})$ // get public transcript (transactions and signatures) on chains A and B 5 : $TX_1 := (TX_A^1, TX_B^1)$ 6 : $TX_0 := (TX_A^1, TX_B^2)$ 7 : $b^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Swap}}}(TX_b)$ 8 : return $(b = b^*)$
$\mathcal{O}_{\text{Swap}}(T^{\text{refund}}, T^{\text{transfer}})$
9 : $(TX_A, TX_B) \leftarrow$ Run honest execution of Π_{Swap} with $(T^{\text{refund}}, T^{\text{transfer}})$ 10 : $TX := (TX_A, TX_B)$ 11 : return TX

Figure 3: Unlinkability game for privacy of atomic swaps.

$\text{Fung}_{\mathcal{A}, \Pi_{\text{Swap}}}(\lambda)$
1 : $b \leftarrow_{\$} \{0, 1\}$ 2 : $tx_0 \leftarrow$ Honest swap transaction (from Π_{Swap}) 3 : $\tau \leftarrow_{\$} \mathcal{T}$ 4 : $tx_1 \leftarrow$ Honest non-swap transaction of type τ 5 : $b^* \leftarrow \mathcal{A}(tx_b)$ 6 : return $(b = b^*)$

Figure 4: Fungibility game for privacy of atomic swaps.

3 Atomic Swaps with Adaptor Signatures

We first introduce the ideal functionality for atomic swaps with timed transactions limited to one party's blockchains (Figure 5 provides an overview). Then we describe and discuss the protocol (Figure 6 provides an overview and Figure 7 provides the full protocol description). We support our protocol with a formal security proof and a privacy analysis.

3.1 Formalization of Functional Requirements

Before diving into the protocol description, we present the ideal functionality the protocol must fulfill. This is necessary in order to prove the security of the scheme. Thyagarajan et al. [TMM22] describe the ideal functionality for an atomic swap where P_0 has n coins and P_1 has m coins. We minimize the requirements for atomic swaps by assuming that only P_1 's blockchain(s) support timed transactions while P_0 's blockchain(s) do not provide this feature. The challenge that arises is that P_0 can refund their coins anytime they like, which needs to be reflected in the ideal functionality. Hence, we need to adapt the ideal functionality of Thyagarajan et al., where both parties have access to a variant of timed transactions. On a high level, to accommodate this requirement, our ideal functionality

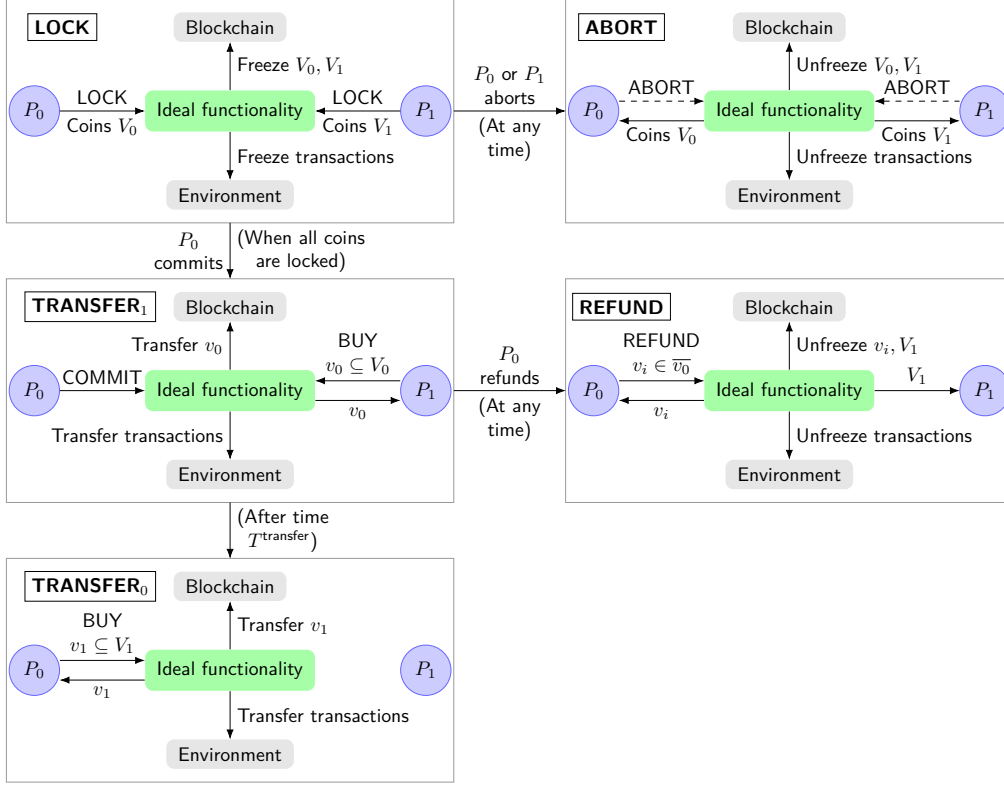


Figure 5: Overview of our ideal functionality. V_0 denotes the set of coins belonging to party P_0 , V_1 the set of coins belonging to party P_1 . v_0 resp. v_1 are subsets of the coin sets V_0 resp. V_1 , with $\bar{v}_0$ being the complement of v_0 ($\bar{v}_0 = V_0 \setminus v_0$) and v_i an element of $\bar{v}_0$.

needs to guarantee two certain conditions. 1. Before P_0 sends COMMIT : If anyone chooses to ABORT , both parties get all their coins back. 2. After P_0 sent COMMIT : If P_0 does not REFUND : P_0 can get all of P_1 's coins and P_1 can get all of P_0 's coins. If P_0 REFUNDs a single coin before a certain time T^{transfer} is reached, P_1 gets all their coins back. To avoid race conditions, the parties agree in advance on time parameters T^{refund} and T^{transfer} with $T^{\text{refund}} < T^{\text{transfer}}$, where T^{transfer} is chosen sufficiently later than T^{refund} . Moreover, P_0 may enter the commit phase at time t only if it holds $t + \Delta < T^{\text{refund}}$, where Δ is a conservatively chosen upper bound on the delay. Intuitively, Δ captures a worst-case upper bound on the time needed for P_0 to post the commit transactions of P_1 's coins (see 3. in Figure 6).

Figure 5 illustrates our proposed ideal functionality on a high level without describing the exact communication that is necessary to provide the signatures, learn the addresses, and control when each coin is spent. We will now detail the ideal functionality. We begin by describing the following two necessary subroutines.

Synchronization and Communication. We ensure synchronisation between parties by utilising an ideal global clock functionality $\mathcal{G}^{\text{clock}}$ according to the definition of Badertscher et al. [Bad+20]. In this model, protocol execution proceeds in rounds. $\mathcal{G}^{\text{clock}}$ only advances once all honest parties have confirmed they are ready. Any party can determine the current round by sending a request to $\mathcal{G}^{\text{clock}}$; therefore, all parties always know the current round. In each round, we assume synchronous communication between parties, which takes place in secure message transmission channels modeled by $\mathcal{F}^{\text{smt}}$ [Can20].

Ideal Blockchain. We model the ledger/blockchain functionality $\mathcal{F}^B$ with an append-only list of transactions. When receiving a transaction with its signature, $\mathcal{F}^B$ examines the validity of both the signature and the transaction (e.g., whether the coin is actually located at pk^{before} and whether pk^{after} is feasible). If both are valid, $\mathcal{F}^B$ adds the transaction to the list. Furthermore, any party and any observer can send a read request to $\mathcal{F}^B$ and receive a full transcript of the transaction list. To maintain simplicity, all blockchains involved in the swap are modeled by the same functionality in distinct instances.

Definition 19 (Ideal functionality). The ideal functionality $\mathcal{F}^{\text{swap}}$ interacts with parties P_0 and P_1 , an ideal blockchain $\mathcal{F}^B$, and the environment $\mathcal{E}$. It is initialized with absolute points in time T^{refund} and T^{transfer} . The functionality receives labeled inputs from the parties, where the label specifies the requested operation (e.g., **lock**, **refund**, **commit**, **redeem**, or **transfer**). Upon receiving such an input, $\mathcal{F}^{\text{swap}}$ processes it according to the interface specification below. We emphasize that $\mathcal{F}^{\text{swap}}$ forwards any transaction/signature pair to $\mathcal{F}^B$ only if the transaction has the prescribed form, the signature is well-formed, the required state conditions are met, and any relevant timeout condition is satisfied.

Initialization. Upon first activation, $\mathcal{F}^{\text{swap}}$ initializes its local state by setting $\text{locked}_0[i] = \text{redeemed}_0[i] = \text{refunded}_0[i] = 0$ for all i , setting $\text{locked}_1[j] = \text{committed}[j] = \text{refunded}_1[j] = \text{transferred}[j] = 0$ for all j , and setting $\text{commit-started} = 0$.

LOCK. In this phase, both parties' coins are locked into joint outputs:

- Upon receiving $(\text{lock}, i, tx_0^{\text{lock},i}, \sigma_0^{\text{lock},i})$ from P_0 , if $tx_0^{\text{lock},i} = (pk_0^{\text{wallet},i}, pk_0^{01,i}, v_0^i)$, if $\text{locked}_0[i] = 0$, and if $\text{commit-started} = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx_0^{\text{lock},i}, \sigma_0^{\text{lock},i})$ to $\mathcal{F}^B$ and sets $\text{locked}_0[i] := 1$.
- Upon receiving $(\text{lock}, j, tx_1^{\text{lock},j}, \sigma_1^{\text{lock},j})$ from P_1 , if $tx_1^{\text{lock},j} = (pk_1^{\text{wallet},j}, pk_1^{10,j}, v_1^j)$, if $\text{locked}_1[j] = 0$, and if $\text{commit-started} = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx_1^{\text{lock},j}, \sigma_1^{\text{lock},j})$ to $\mathcal{F}^B$ and sets $\text{locked}_1[j] := 1$.

ABORT. At any time during the LOCK phase, one of the parties may choose to abort, which enables both parties to refund all their coins:

- Upon receiving $(\text{refund}, i, tx^{\text{refund},0,i}, \sigma^{\text{refund},0,i})$ from P_0 , if $tx^{\text{refund},0,i} = (pk_0^{01,i}, pk_0^{\text{refund},i}, v_0^i)$, if $\text{commit-started} = 0$, if $\text{locked}_0[i] = 1$, if $\text{redeemed}_0[i] = 0$, and if $\text{refunded}_0[i] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{refund},0,i}, \sigma^{\text{refund},0,i})$ to $\mathcal{F}^B$ and sets $\text{refunded}_0[i] := 1$.
- Upon receiving $(\text{refund}, j, tx^{\text{refund},1,T,j}, \sigma^{\text{refund},1,T,j})$ from P_1 , if $tx^{\text{refund},1,T,j} = (pk_1^{10,j}, pk_1^{\text{refund},j}, v_1^j, T^{\text{refund}})$, if $\text{commit-started} = 0$, if $\text{locked}_1[j] = 1$, and if $\text{refunded}_1[j] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{refund},1,T,j}, \sigma^{\text{refund},1,T,j})$ to $\mathcal{F}^B$ and sets $\text{refunded}_1[j] := 1$.

TRANSFER₁. Once all coins are locked (but before time T^{refund}), P_0 can choose to commit, which enables P_1 to redeem P_0 's coins:

- Upon receiving $(\text{commit}, j, tx^{10,j}, \sigma_1^{10,j})$ from P_0 , if $tx^{10,j} = (pk_1^{10,j}, pk_1^{\text{delay},j}, v_1^j)$, if $\text{locked}_0[i] = 1$ for all i , if $\text{locked}_1[j'] = 1$ for all j' , if at time t a sufficient amount of time Δ before T^{refund} remains, i.e., $t + \Delta < T^{\text{refund}}$, and if $\text{committed}[j] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{10,j}, \sigma_1^{10,j})$ to $\mathcal{F}^B$ and sets $\text{committed}[j] := 1$ and $\text{commit-started} := 1$.
- Upon receiving $(\text{redeem}, i, tx^{01,i}, \sigma^{01,i})$ from P_1 , if $tx^{01,i} = (pk_1^{01,i}, pk_1^{\text{transfer},i}, v_0^i)$, if $\text{commit-started} = 1$, if $\text{redeemed}_0[i] = 0$, and if $\text{refunded}_0[i] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{01,i}, \sigma^{01,i})$ to $\mathcal{F}^B$ and sets $\text{redeemed}_0[i] := 1$.

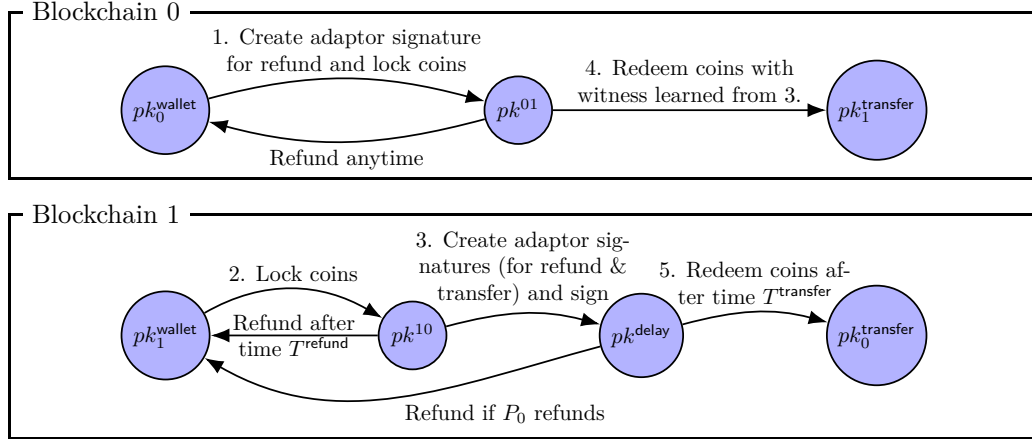


Figure 6: Overview to our approach with timelocks only on blockchain 1.

REFUND. At any time during the TRANSFER_1 phase, P_0 may choose to refund a coin that has not yet been redeemed by P_1 , which enables P_1 to refund all their coins:

- Upon receiving $(\text{refund}, i, tx^{\text{refund}, 0, i}, \sigma^{\text{refund}, 0, i})$ from P_0 , if $tx^{\text{refund}, 0, i} = (pk^{01, i}, pk_0^{\text{refund}, i}, v_0^i)$, if $\text{commit-started} = 1$, if $\text{locked}_0[i] = 1$, if $\text{redeemed}_0[i] = 0$, and if $\text{refunded}_0[i] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{refund}, 0, i}, \sigma^{\text{refund}, 0, i})$ to $\mathcal{F}^B$ and sets $\text{refunded}_0[i] := 1$.
- Upon receiving $(\text{refund}, j, tx, \sigma)$ from P_1 , if there exists an index i such that $\text{refunded}_0[i] = 1$, then $\mathcal{F}^{\text{swap}}$ proceeds as follows:
 - If $\text{committed}[j] = 0$, if $\text{locked}_1[j] = 1$, if $\text{refunded}_1[j] = 0$, and if $tx = tx^{\text{refund}, 1, T, j} = (pk^{10, j}, pk_1^{\text{refund}, j}, v_1^j, T^{\text{refund}})$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{refund}, 1, T, j}, \sigma^{\text{refund}, 1, T, j})$ to $\mathcal{F}^B$ and sets $\text{refunded}_1[j] := 1$.
 - If $\text{committed}[j] = 1$, if $\text{locked}_1[j] = 1$, if $\text{refunded}_1[j] = 0$, if $\text{transferred}[j] = 0$, and if $tx = tx^{\text{refund}, 1, j} = (pk^{\text{delay}, j}, pk_1^{\text{refund}, j}, v_1^j)$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{refund}, 1, j}, \sigma^{\text{refund}, 1, j})$ to $\mathcal{F}^B$ and sets $\text{refunded}_1[j] := 1$.

TRANSFER₀. If P_0 does not refund, then P_0 can redeem P_1 's coins after the delay T^{transfer} :

- Upon receiving $(\text{transfer}, j, tx^{\text{transfer}, T, j}, \sigma^{\text{transfer}, T, j})$ from P_0 , if $tx^{\text{transfer}, T, j} = (pk^{\text{delay}, j}, pk_0^{\text{transfer}, j}, v_1^j, T^{\text{transfer}})$, if $\text{committed}[j] = 1$, if $\text{refunded}_1[j] = 0$, if $\text{transferred}[j] = 0$, and if for all indices i it holds that $\text{refunded}_0[i] = 0$, then $\mathcal{F}^{\text{swap}}$ forwards $(tx^{\text{transfer}, T, j}, \sigma^{\text{transfer}, T, j})$ to $\mathcal{F}^B$ and sets $\text{transferred}[j] := 1$.

Remark 3. For readability, we omit the explicit operation labels of inputs to $\mathcal{F}^{\text{swap}}$ in the following whenever the context for a transaction/signature pair, including the corresponding superscripts and indices, already determines them. For example, when we write that a party sends or receives $tx_0^{\text{lock}, i}$ and $\sigma_0^{\text{lock}, i}$, this is to be understood as shorthand for the labeled input $(\text{lock}, i, tx_0^{\text{lock}, i}, \sigma_0^{\text{lock}, i})$. More generally, such notation is to be understood as shorthand for the corresponding labeled input in Definition 19. This is purely a notational simplification: $\mathcal{F}^{\text{swap}}$ still processes each input according to the interface above and ignores malformed input.

3.2 Protocol Description

Building on Hoenisch et al. [Hoe+22] as well as Thyagarajan et al. [TMM22], we present an atomic swap protocol that eliminates time-locked transactions for one party. We provide

Initialization:

- P_0 owns coins $v_0^1, \dots, v_0^n$ at addresses $pk_0^{\text{wallet},1}, \dots, pk_0^{\text{wallet},n}$ with $sk_0^{\text{wallet},1}, \dots, sk_0^{\text{wallet},n}$
- P_1 owns coins $v_1^1, \dots, v_1^m$ at addresses $pk_1^{\text{wallet},1}, \dots, pk_1^{\text{wallet},m}$ with $sk_1^{\text{wallet},1}, \dots, sk_1^{\text{wallet},m}$
- P_0 and P_1 agree on time parameters T^{refund} and T^{transfer} and Δ

Swap protocol:

1. Preparation and locking of P_0 's coins with $1 \leq i \leq n$:

- P_0 chooses $(y^{\text{refund}}, Y^{\text{refund}}) \in R$ and sends P_1 only the statement Y^{refund}
 - P_0 and P_1 run $(pk^{01,i}, sk_0^{01,i}, sk_1^{01,i}) \leftarrow \text{MuKGen}(1^\lambda)$ to generate joint addresses $pk^{01,i}$ to lock P_0 's coins at, with each party getting a share of the corresponding secret key
 - P_0 and P_1 create adaptor signatures $\tilde{\sigma}^{\text{refund},0,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}})$ for P_0 's refund transactions $tx^{\text{refund},0,i} = (pk^{01,i}, pk_0^{\text{refund},i}, v_0^i)$
 - Both parties check $1 = \text{MuPVrfy}(pk^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}}, \tilde{\sigma}^{\text{refund},0,i})$; else both abort
 - P_0 signs $tx_0^{\text{lock},i} = (pk_0^{\text{wallet},i}, pk^{01,i}, v_0^i)$ with $sk_0^{\text{wallet},i}$ to lock P_0 's coins
- $\Rightarrow P_0$ can refund anytime by posting $\sigma^{\text{refund},0,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},0,i}, y^{\text{refund}})$ on $tx^{\text{refund},0,i}$

2. Locking of P_1 's coins with $1 \leq j \leq m$:

- P_0 and P_1 run $(pk^{10,j}, sk_0^{10,j}, sk_1^{10,j}) \leftarrow \text{MuKGen}(1^\lambda)$ to generate joint addresses $pk^{10,j}$ to lock P_1 's coins at, with each party getting a share of the corresponding secret key
 - P_0 and P_1 run $\sigma^{\text{refund},1,T,j} \leftarrow \text{MuSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{\text{refund},1,T,j})$ to sign timed refund transactions $tx^{\text{refund},1,T,j} = (pk^{10,j}, pk_1^{\text{refund},j}, v_1^j, T^{\text{refund}})$
 - P_1 checks $1 = \text{MuVrfy}(pk^{10,j}, tx^{\text{refund},1,T,j}, \sigma^{\text{refund},1,T,j})$; else P_1 aborts
 - P_1 signs $tx_1^{\text{lock},j} = (pk_1^{\text{wallet},j}, pk^{10,j}, v_1^j)$ with $sk_1^{\text{wallet},j}$ to lock P_1 's coins
- $\Rightarrow P_1$ can refund after T^{refund}

3. Creation of adaptor signatures and moving P_1 's coins with $1 \leq i \leq n$ and $1 \leq j \leq m$:

- P_0 and P_1 run $(pk^{\text{delay},j}, sk_0^{\text{delay},j}, sk_1^{\text{delay},j}) \leftarrow \text{MuKGen}(1^\lambda)$ to generate joint addresses $pk^{\text{delay},j}$ to move P_1 's coins to, with each party getting a share of the corresponding secret key
 - P_0 and P_1 create adaptor signatures $\tilde{\sigma}^{\text{refund},1,j} \leftarrow \text{MuPSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{refund},1,j}, Y^{\text{refund}})$ for P_1 's refund transactions $tx^{\text{refund},1,j} = (pk^{\text{delay},j}, pk_1^{\text{refund},j}, v_1^j)$
 - P_1 checks $1 = \text{MuPVrfy}(pk^{\text{delay},j}, tx^{\text{refund},1,j}, Y^{\text{refund}}, \tilde{\sigma}^{\text{refund},1,j})$; else P_1 aborts and refunds
- $\Rightarrow$ Once (part of) P_1 's coins are at $pk^{\text{delay},j}$, P_1 can refund them—if P_0 refunds any of their coins with $\sigma^{\text{refund},0,i}$ —by extracting the witness $y^{\text{refund}} \leftarrow \text{MuExt}(\sigma^{\text{refund},0,i}, \tilde{\sigma}^{\text{refund},0,i}, Y^{\text{refund}})$ and then posting $\sigma^{\text{refund},1,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},1,j}, y^{\text{refund}})$ on $tx^{\text{refund},1,j}$
- P_0 chooses $(y^{\text{transfer}}, Y^{\text{transfer}}) \in R$ and sends P_1 only the statement Y^{transfer}
 - P_0 and P_1 create adaptor signatures $\tilde{\sigma}^{01,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{01,i}, Y^{\text{transfer}})$ for transactions $tx^{01,i} = (pk^{01,i}, pk_1^{\text{transfer},i}, v_0^i)$ that transfer P_0 's coins to P_1
 - P_1 checks $1 = \text{MuPVrfy}(pk^{01,i}, tx^{01,i}, Y^{\text{transfer}}, \tilde{\sigma}^{01,i})$; else P_1 aborts and refunds after T^{refund}
 - P_0 and P_1 create adaptor signatures $\tilde{\sigma}^{10,j} \leftarrow \text{MuPSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{10,j}, Y^{\text{transfer}})$ for the ensuing transactions $tx^{10,j} = (pk^{10,j}, pk^{\text{delay},j}, v_1^j)$
 - Both parties check $1 = \text{MuPVrfy}(pk^{10,j}, tx^{10,j}, Y^{\text{transfer}}, \tilde{\sigma}^{10,j})$; else both abort and refund
 - P_0 and P_1 run $\sigma^{\text{transfer},T,j} \leftarrow \text{MuSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{transfer},T,j})$ to sign $tx^{\text{transfer},T,j} = (pk^{\text{delay},j}, pk_0^{\text{transfer},j}, v_1^j, T^{\text{transfer}})$ that give P_1 's coins to P_0
 - P_0 checks $1 = \text{MuVrfy}(pk^{\text{delay},j}, tx^{\text{transfer},T,j}, \sigma^{\text{transfer},T,j})$; else P_0 aborts and refunds
 - P_0 checks that at time t a sufficient amount of time Δ before T^{refund} remains, i.e., $t + \Delta < T^{\text{refund}}$; else P_0 aborts and refunds
 - P_0 moves P_1 's coins by posting $\sigma^{10,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{10,j}, y^{\text{transfer}})$ on $tx^{10,j}$

4. Redemption of P_0 's coins with $1 \leq i \leq n$:

- P_1 redeems P_0 's coins by extracting the witness $y^{\text{transfer}} \leftarrow \text{MuExt}(\sigma^{10,j}, \tilde{\sigma}^{10,j}, Y^{\text{transfer}})$ and then posting $\sigma^{01,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{01,i}, y^{\text{transfer}})$ on $tx^{01,i}$

5. Redemption of P_1 's coins with $1 \leq j \leq m$:

- After T^{transfer} , transactions $tx^{\text{transfer},T,j}$ (signed in step 3) become valid, giving P_1 's coins to P_0

Figure 7: Formal description of our atomic swap protocol.

LOCK P_0	
1 : $\mathbf{P}_0$	$\mathbf{P}_1$
2 : Init: $v_0^i, pk_0^{\text{wallet},i}, sk_0^{\text{wallet},i}, 1 \leq i \leq n$	
3 : Choose $(y^r, Y^r) \in R$	$\xrightarrow{Y^r}$
4 : $(pk_0^{01,i}, sk_0^{01,i}) \leftarrow \text{KGen}(1^\lambda)$	$\xrightarrow{pk_0^{01,i}}$
5 :	$\xleftarrow{pk_1^{01,i}} (pk_1^{01,i}, sk_1^{01,i}) \leftarrow \text{KGen}(1^\lambda)$
6 : $pk_0^{01,i} \leftarrow \text{KAgg}(pk_0^{01,i}, pk_1^{01,i})$	$pk_0^{01,i} \leftarrow \text{KAgg}(pk_0^{01,i}, pk_1^{01,i})$
7 : $(pk_0^{r,i}, sk_0^{r,i}) \leftarrow \text{KGen}(1^\lambda)$	
8 : $tx^{r,0,i} = (pk_0^{01,i}, pk_0^{r,i}, v_0^i)$	$\xrightarrow{tx^{r,0,i}}$
9 : $\tilde{\sigma}_0^{r,0,i} \leftarrow \text{pSign}(sk_0^{01,i}, tx^{r,0,i}, Y^r)$	$\xrightarrow{\tilde{\sigma}_0^{r,0,i}}$
10 :	$\xleftarrow{\tilde{\sigma}_1^{r,0,i}} \tilde{\sigma}_1^{r,0,i} \leftarrow \text{pSign}(sk_1^{01,i}, tx^{r,0,i}, Y^r)$
11 : $\tilde{\sigma}^{r,0,i} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{r,0,i}, \tilde{\sigma}_1^{r,0,i})$	$\tilde{\sigma}^{r,0,i} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{r,0,i}, \tilde{\sigma}_1^{r,0,i})$
12 : if $\neg \text{MuPVrfy}(pk_0^{01,i}, tx^{r,0,i}, Y^r, \tilde{\sigma}^{r,0,i})$:	if $\neg \text{MuPVrfy}(pk_1^{01,i}, tx^{r,0,i}, Y^r, \tilde{\sigma}^{r,0,i})$:
13 : abort	abort
14 : $tx_0^{\text{lock},i} = (pk_0^{\text{wallet},i}, pk_0^{01,i}, v_0^i)$	
15 : $\sigma_0^{\text{lock},i} \leftarrow \text{Sign}(sk_0^{\text{wallet},i}, tx_0^{\text{lock},i})$	

LOCK P_1	
1 : $\mathbf{P}_0$	$\mathbf{P}_1$
2 :	Init: $v_1^j, pk_1^{\text{wallet},j}, sk_1^{\text{wallet},j}, 1 \leq j \leq m$
3 : $(pk_0^{10,j}, sk_0^{10,j}) \leftarrow \text{KGen}(1^\lambda)$	$\xrightarrow{pk_0^{10,j}}$
4 :	$\xleftarrow{pk_1^{10,j}} (pk_1^{10,j}, sk_1^{10,j}) \leftarrow \text{KGen}(1^\lambda)$
5 : $pk_0^{10,j} \leftarrow \text{KAgg}(pk_0^{10,j}, pk_1^{10,j})$	$pk_0^{10,j} \leftarrow \text{KAgg}(pk_0^{10,j}, pk_1^{10,j})$
6 :	$(pk_1^{r,j}, sk_1^{r,j}) \leftarrow \text{KGen}(1^\lambda)$
7 :	$\xleftarrow{tx^{r,1,T,j}} tx^{r,1,T,j} = (pk_0^{10,j}, pk_1^{r,j}, v_1^j, T^r)$
8 : $\sigma_0^{r,1,T,j} \leftarrow \text{Sign}(sk_0^{10,j}, tx^{r,1,T,j})$	$\xrightarrow{\sigma_0^{r,1,T,j}}$
9 :	$\sigma_1^{r,1,T,j} \leftarrow \text{Sign}(sk_1^{10,j}, tx^{r,1,T,j})$
10 :	$\sigma^{r,1,T,j} \leftarrow \text{SigAgg}(\sigma_0^{r,1,T,j}, \sigma_1^{r,1,T,j})$
11 :	if $\neg \text{MuVrfy}(pk_0^{10,j}, tx^{r,1,T,j}, \sigma^{r,1,T,j})$:
12 :	abort
13 :	$tx_1^{\text{lock},j} = (pk_1^{\text{wallet},j}, pk_0^{10,j}, v_1^j)$
14 :	$\sigma_1^{\text{lock},j} \leftarrow \text{Sign}(sk_1^{\text{wallet},j}, tx_1^{\text{lock},j})$

Figure 8: Interactive overview of procedures for locking P_0 's coins (above) and P_1 's coins (below). Here, we shorten the notation with t for transfer, r for refund, and d for delay.

an overview of our protocol in Figure 6, present the full description Π_{Swap} in Figure 7, and

COMMIT $P_1 \rightarrow P_0$		
1 :	P_0	P_1
2 :	$(pk_0^{d,j}, sk_0^{d,j}) \leftarrow \text{KGen}(1^\lambda)$	$\xrightarrow{pk_0^{d,j}}$
3 :		$\xleftarrow{pk_1^{d,j}} (pk_1^{d,j}, sk_1^{d,j}) \leftarrow \text{KGen}(1^\lambda)$
4 :	$pk^{d,j} \leftarrow \text{KAgg}(pk_0^{d,j}, pk_1^{d,j})$	$pk^{d,j} \leftarrow \text{KAgg}(pk_0^{d,j}, pk_1^{d,j})$
5 :		$\xleftarrow{tx^{r,1,j}} tx^{r,1,j} = (pk^{d,j}, pk_1^{r,j}, v_1^j)$
6 :	$\tilde{\sigma}_0^{r,1,j} \leftarrow \text{pSign}(sk_0^{d,j}, tx^{r,1,j}, Y^r)$	$\xrightarrow{\tilde{\sigma}_0^{r,1,j}}$
7 :		$\tilde{\sigma}_1^{r,1,j} \leftarrow \text{pSign}(sk_1^{d,j}, tx^{r,1,j}, Y^r)$
8 :		$\tilde{\sigma}^{r,1,j} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{r,1,j}, \tilde{\sigma}_1^{r,1,j})$
9 :		if $\neg \text{MuPVrfy}(pk^{d,j}, tx^{r,1,j}, Y^r, \tilde{\sigma}^{r,1,j})$:
10 :		abort
11 :	Choose $(y^t, Y^t) \in R$	$\xrightarrow{Y^t}$
12 :		$(pk_1^{t,i}, sk_1^{t,i}) \leftarrow \text{KGen}(1^\lambda)$
13 :		$\xleftarrow{tx^{01,i}} tx^{01,i} = (pk^{01,i}, pk_1^{t,i}, v_0^i)$
14 :	$\tilde{\sigma}_0^{01,i} \leftarrow \text{pSign}(sk_0^{01,i}, tx^{01,i}, Y^t)$	$\xrightarrow{\tilde{\sigma}_0^{01,i}}$
15 :		$\tilde{\sigma}_1^{01,i} \leftarrow \text{pSign}(sk_1^{01,i}, tx^{01,i}, Y^t)$
16 :		$\tilde{\sigma}^{01,i} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{01,i}, \tilde{\sigma}_1^{01,i})$
17 :		if $\neg \text{MuPVrfy}(pk^{01,i}, tx^{01,i}, Y^t, \tilde{\sigma}^{01,i})$:
18 :		abort
19 :	$tx^{10,j} = (pk^{10,j}, pk^{d,j}, v_1^j)$	$tx^{10,j} = (pk^{10,j}, pk^{d,j}, v_1^j)$
20 :	$\tilde{\sigma}_0^{10,j} \leftarrow \text{pSign}(sk_0^{10,j}, tx^{10,j}, Y^t)$	$\xrightarrow{\tilde{\sigma}_0^{10,j}}$
21 :		$\xleftarrow{\tilde{\sigma}_1^{10,j}} \tilde{\sigma}_1^{10,j} \leftarrow \text{pSign}(sk_1^{10,j}, tx^{10,j}, Y^t)$
22 :	$\tilde{\sigma}^{10,j} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{10,j}, \tilde{\sigma}_1^{10,j})$	$\tilde{\sigma}^{10,j} \leftarrow \text{pSigAgg}(\tilde{\sigma}_0^{10,j}, \tilde{\sigma}_1^{10,j})$
23 :	if $\neg \text{MuPVrfy}(pk^{10,j}, tx^{10,j}, Y^t, \tilde{\sigma}^{10,j})$:	if $\neg \text{MuPVrfy}(pk^{10,j}, tx^{10,j}, Y^t, \tilde{\sigma}^{10,j})$:
24 :	abort	abort
25 :	$(pk_0^{t,j}, sk_0^{t,j}) \leftarrow \text{KGen}(1^\lambda)$	
26 :	$tx^{t,T,j} = (pk^{d,j}, pk_0^{t,j}, v_1^j, T^t)$	$\xrightarrow{tx^{t,T,j}}$
27 :	$\sigma_0^{t,T,j} \leftarrow \text{Sign}(sk_0^{d,j}, tx^{t,T,j})$	
28 :		$\xleftarrow{\sigma_1^{t,T,j}} \sigma_1^{t,T,j} \leftarrow \text{Sign}(sk_1^{d,j}, tx^{t,T,j})$
29 :	$\sigma^{t,T,j} \leftarrow \text{SigAgg}(\sigma_0^{t,T,j}, \sigma_1^{t,T,j})$	
30 :	if $\neg \text{MuVrfy}(pk^{d,j}, tx^{t,T,j}, \sigma^{t,T,j})$: abort	
31 :	if not $T^{\text{current}} << T^r$: abort	
32 :	$\sigma^{10,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{10,j}, y^t)$	

Figure 9: Interactive overview of the commit procedure. When all coins are locked, P_0 commits. Then P_0 gets P_1 's coins after T^t (which is agreed-upon beforehand). Here, we shorten the notation with t for transfer, r for refund, and d for delay.

TRANSFER $P_0 \rightarrow P_1$	REFUND P_0	REFUND P_1
1 : $\mathbf{P}_1$	$\mathbf{P}_0$	$\mathbf{P}_1$
2 : $y^t \leftarrow \text{MuExt}(\sigma^{10,j}, \tilde{\sigma}^{10,j}, Y^t)$	$\sigma^{r,0,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{r,0,i}, y^r)$	$y^r \leftarrow \text{MuExt}(\sigma^{r,0,i}, \tilde{\sigma}^{r,0,i}, Y^r)$
3 : $\sigma^{01,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{01,i}, y^t)$		$\sigma^{r,1,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{r,1,j}, y^r)$

Figure 10: Left: After P_0 committed, P_1 redeems P_0 's coins. Middle: P_0 can refund anytime after locking their coins. Right: If P_0 refunds, P_1 can also refund their committed coins. Here, we shorten the notation with t for transfer, and r for refund.

give an overview of the interaction between the parties in Figures 8 to 10. We now provide a detailed explanation and discussion of our approach. For readability, we first explain the protocol concept with only one blockchain per party, and then provide a discussion and the full protocol where each party's coins can be located on multiple blockchains. W.l.o.g. we assume that P_0 operates on a blockchain without timers while P_1 owns coins on a blockchain that provides timed transactions. The protocol proceeds as follows. Before the coins are locked, P_0 and P_1 together create an adaptor signature $\tilde{\sigma}^{\text{refund},0}$ for the refund transaction of P_0 using a statement Y^{refund} where only P_0 knows the witness y^{refund} . Subsequently, P_0 locks their coins at the joint address pk^{01} where the corresponding sk^{01} is a combination of both parties' secret keys, sk_0^{01} and sk_1^{01} . As P_0 knows the adaptor signature $\tilde{\sigma}^{\text{refund},0}$ and the associated witness, P_0 can refund their coins anytime. Then, P_1 locks their coins at the joint address pk^{10} where the aggregate sk^{10} is a combination of both parties' secret keys, sk_0^{10} and sk_1^{10} . However, after a certain time T^{refund} P_1 can refund their coins. Before proceeding, P_0 and P_1 together create another adaptor signature $\tilde{\sigma}^{\text{refund},1}$ using the same statement Y^{refund} for the refund transaction from the joint address pk^{delay} back to P_1 . Afterwards, P_0 and P_1 construct an adaptor signature $\tilde{\sigma}^{01}$ using a different statement Y^{transfer} (where P_0 again knows the witness y^{transfer}) for the transaction tx^{01} that transfers P_0 's coins to P_1 . With the same statement Y^{transfer} , P_0 and P_1 create an adaptor signature $\tilde{\sigma}^{10}$ for the ensuing transaction moving P_1 's coins from pk^{10} to another joint address pk^{delay} . Again, P_1 knows sk_1^{delay} and P_0 knows sk_0^{delay} . P_0 then posts this transaction, authorizing it with the signature learnt from $\tilde{\sigma}^{10}$ and y^{transfer} . At any time, in case P_0 refunds, P_1 is able to extract the witness y^{refund} from $\tilde{\sigma}^{\text{refund},0}$ and therefore refund their own coins using $\tilde{\sigma}^{\text{refund},1}$. Consequently, with the witness y^{transfer} learned from $\tilde{\sigma}^{10}$ and the corresponding valid signature, P_1 can redeem P_0 's coins using $\tilde{\sigma}^{01}$. Finally, after a time T^{transfer} , P_0 can redeem P_1 's coins.

Discussion. To provide versatility, our protocol allows each party's coins to reside on different blockchains. Thus, we assume P_0 holds n coins and P_1 holds m coins on potentially distinct chains, where some of P_0 's chains may lack timed transactions. To achieve this, we assume that all blockchains involved in our protocol provide a (but not necessarily the same) digital signature scheme according to Definition 1. In particular, we assume all blockchains to support two-party signatures according to Definition 3: Either the blockchain offers a two-party signature functionality (as is the case for Bitcoin [BDN18], Ethereum [BW24], and others), or the underlying signature scheme of the blockchain must be inherently modifiable into two-party signatures (as can be done with Schnorr and ECDSA signatures [TZ23]). We require that the corresponding two-party adaptor signature schemes all work with the same hard relation $(y, Y) \in R$. The order of creating the adaptor signatures is important! If $\tilde{\sigma}^{\text{refund},1}$ was created after $\tilde{\sigma}^{10}$, P_0 could transfer P_1 's coins to pk^{10} prematurely using $\tilde{\sigma}^{10}$ and then refund P_0 's coins without P_1 being able to refund as well. Additionally, if $\tilde{\sigma}^{10}$ was generated before $\tilde{\sigma}^{01}$, P_0 would again be able to transfer P_1 's coins to pk^{10} prematurely, giving P_0 those coins after T^{transfer} , but P_1 could

not post a valid signature on tx^{01} and therefore never get P_0 's coins. Furthermore, to avoid race conditions, we require the absolute time parameter T^{transfer} to be set sufficiently later than T^{refund} . Otherwise, P_0 could commit P_1 's coins shortly before T^{refund} and then refund (some of) P_0 's coins, creating a race for P_1 's coins where P_1 tries to refund them and P_0 tries to redeem them after T^{transfer} . Finally, the way in which P_0 and P_1 decide which joint addresses to lock the coins at when using MuKGen is highly dependent on the two-party signature implementation used. We assume that it can either be done by a blockchain functionality or is part of the two-party signature scheme. With Schnorr signatures for example, pk is simply $pk_0 + pk_1$.

3.3 Security Analysis

We prove that our scheme fulfills the security requirements of correctness and soundness as specified in Definition 16. For this, we first argue that the ideal functionality proposed in Section 3.1 satisfies these requirements. Then, we show that our protocol is indistinguishable from the ideal functionality and therefore also satisfies the security requirements.

Theorem 1 (Security of $\mathcal{F}^{\text{swap}}$). *The ideal functionality $\mathcal{F}^{\text{swap}}$ stated in Definition 19 meets the security requirements of correctness and soundness according to Definition 16.*

Proof. The correctness follows directly from the construction. We observe that the ideal functionality ends with both parties having exchanged their coins if they behave honestly and neither party refunds: P_0 locks n coins and P_1 locks m coins. Subsequently, P_0 commits all m coins of P_1 , which enables P_1 to redeem all n of P_0 's coins. Finally, P_0 can redeem all m coins of P_1 after T^{transfer} .

Regarding soundness, we need to look at two cases depending on which party is honest. For this, we assume that no adversary can force the honest party to go offline. In Case 1, P_0 is honest and P_1 is corrupted. P_0 locks their coins and refunds all of them if P_1 deviates from the LOCK protocol or takes too long. Otherwise, P_0 commits all of P_1 's coins and is able to redeem them after T^{transfer} . Therefore, an honest P_0 either gets all their coins back or gets all of P_1 's coins. In Case 2, P_1 is honest and P_0 is corrupted. P_1 locks their coins and refunds all of them after T^{refund} if P_1 deviates from the LOCK protocol. Otherwise, after P_0 has posted the first commit transaction, P_1 starts to claim P_0 's coins. If P_0 does not refund any of their coins during this process, P_1 is able to claim all of P_0 's coins. On the other hand, if P_0 chooses to refund a single coin at this stage, P_1 can get all their own coins back. Therefore, an honest P_1 always gets either all P_0 's coins or all of their own coins back. $\square$

Indistinguishability. In order to prove security, we need to prove indistinguishability between this ideal functionality and our protocol according to the Universal Composability framework by Canetti [Can20]. The Universal Composability framework allows the use of a so-called simulator that masks the ideal functionality for the distinguisher. Essentially, we require that the probability of distinguishing the interaction with our protocol and the interaction with the simulator combined with the ideal functionality is negligible.

Definition 20 (Universal Composability [TMM22]). A protocol τ UC-realizes an ideal functionality F if for any PPT adversary $\mathcal{A}$ there exists a simulator $\mathcal{S}$ such that for any environment $\mathcal{E}$ the ensembles $\text{EXEC}_{\tau, \mathcal{A}, \mathcal{E}}$ and $\text{EXEC}_{F, \mathcal{S}, \mathcal{E}}$ are computationally indistinguishable.

To be precise, we use the UC framework with global subroutines (UCGS) proposed by Badertscher et al. [Bad+20], because the original UC framework only supports local/isolated setups, but a blockchain constitutes a global component that is shared by multiple protocols. UCGS allows the use of global subroutines, but still captures the security in plain UC and is therefore ideal for our proof.

On the one hand, we have the real-world protocol where P_0 , P_1 and the blockchain all interact with each other and the environment sees transactions on the blockchain and anything the parties choose to share. However, we assume that the environment cannot see the communication between P_0 and P_1 if none of the parties intentionally leaks it. In the ideal-world execution on the other hand, the ideal functionality, the simulator and the environment communicate with each other, but only the ideal functionality interacts with the blockchain. In this, the environment is grouped with the dishonest parties because they form the distinguisher/adversary. Similarly, honest parties are emulated by the simulator as both the ideal functionality and the adversary expect to communicate with them.

Theorem 2 (Indistinguishability of $\mathcal{F}^{\text{swap}}$ and Π_{Swap}). *The protocol Π_{Swap} UC-emulates the ideal functionality $\mathcal{F}^{\text{swap}}$ according to Definition 20.*

Proof. For our proof, we use a static corruption model [DN14] where it is known beforehand which party the adversary corrupts. This gives three cases. In case 1, both parties are honest and the environment is malicious. In this case, the environment wants to distinguish between the information it gets from the blockchain in the real world and the information it gets from the ideal functionality. Essentially, this case proves that our protocol does not leak relevant information if both parties are honest. In case 2, P_0 is honest and P_1 is malicious. Now, P_1 wants to distinguish between the execution of the real-world protocol and the execution of the ideal functionality masked by the simulator. This case guarantees that our protocol does not allow a malicious P_1 to gain an advantage over an honest P_0 . In case 3, P_1 is honest and P_0 is malicious. Analogous to the second case, P_0 wants to distinguish between the execution of the real-world protocol and the execution of the ideal functionality masked by the simulator. This case guarantees that our protocol does not allow a malicious P_0 to gain an advantage over an honest P_1 . We exclude the case in which both parties are dishonest, as security properties are only meaningful in the presence of at least one honest participant.

We now build simulators for each of the three cases and argue that they ensure the indistinguishability from Definition 20. For this, we assume that honest parties act in their best interest. Also, we abbreviate the sets of coins by using $V_0 = v_0^1, \dots, v_0^n$ and $V_1 = v_1^1, \dots, v_1^m$, and denote the sets of keys analogously.

Case 1. both parties are honest. In this case, the simulator is straightforward, as it forwards the transactions and signatures from the ideal functionality to the environment. In the real-world protocol, neither of the honest parties communicates with the environment, so the environment can only see whenever a blockchain transaction with a corresponding signature is posted. Therefore, the environment's view is the same in the real world and the ideal world. It is of course necessary to prove that the transactions that are posted are the same in both cases—but we argue that this is shown in cases 2 and 3, because if the indistinguishability is given for P_0 (case 3) and P_1 (case 2), the transactions that are possible must be identical in the real protocol and the ideal functionality.

Case 2. P_0 is honest and P_1 is malicious. We describe the interaction of the simulator $\mathcal{S}$ with the adversary $\mathcal{A}$ and the ideal functionality. $\mathcal{S}$ is initialized with V_0 , PK_0^{wallet} , SK_0^{wallet} and $\mathcal{A}$ with V_1 , PK_1^{wallet} , SK_1^{wallet} , and both agree on T^{refund} , T^{transfer} and the coins to be swapped.

- LOCK:**
- $\mathcal{S}$ chooses $(y^{\text{refund}}, Y^{\text{refund}}) \in R$ and sends Y^{refund} to $\mathcal{A}$
 - $\mathcal{S}$ and $\mathcal{A}$ run $(pk^{01,i}, sk_0^{01,i}, sk_1^{01,i}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_0^{01,i}$ and $\mathcal{A}$ gets $sk_1^{01,i}$
 - $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{\text{refund},0,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}})$
 - $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}}, \tilde{\sigma}^{\text{refund},0,i})$; else $\mathcal{S}$ executes ABORT
 - $\mathcal{S}$ sends $tx_0^{\text{lock},i}$ with $\sigma_0^{\text{lock},i} \leftarrow \text{Sign}(sk_0^{\text{wallet},i}, tx_0^{\text{lock},i})$ to $\mathcal{F}^{\text{swap}}$
 - $\mathcal{S}$ and $\mathcal{A}$ run $(pk^{10,j}, sk_0^{10,j}, sk_1^{10,j}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_0^{10,j}$ and $\mathcal{A}$ gets $sk_1^{10,j}$

- $\mathcal{S}$ and $\mathcal{A}$ run $\sigma^{\text{refund},1,T,j} \leftarrow \text{MuSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{\text{refund},1,T,j})$
- $\mathcal{S}$ waits until $\mathcal{F}^{\text{swap}}$ has posted $tx_1^{\text{lock},j}$ with $\sigma_1^{\text{lock},j}$ from $\mathcal{A}$ and executes ABORT if the lock addresses differ from $pk^{10,j}$
- Run $(pk^{\text{delay},j}, sk_0^{\text{delay},j}, sk_1^{\text{delay},j}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_0^{\text{delay},j}$ and $\mathcal{A}$ gets $sk_1^{\text{delay},j}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{\text{refund},1,j} \leftarrow \text{MuPSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{refund},1,j}, Y^{\text{refund}})$
- $\mathcal{S}$ chooses $(y^{\text{transfer}}, Y^{\text{transfer}}) \in R$ and sends Y^{transfer} to $\mathcal{A}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{01,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{01,i}, Y^{\text{transfer}})$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{10,j} \leftarrow \text{MuPSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{10,j}, Y^{\text{transfer}})$
- $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{10,j}, tx^{10,j}, Y^{\text{transfer}}, \tilde{\sigma}^{10,j})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ and $\mathcal{A}$ run $\sigma^{\text{transfer},T,j} \leftarrow \text{MuSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{transfer},T,j})$
- $\mathcal{S}$ checks $1 = \text{MuVrfy}(pk^{\text{delay},j}, tx^{\text{transfer},T,j}, \sigma^{\text{transfer},T,j})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ checks that a sufficient amount of time before T^{refund} remains; else $\mathcal{S}$ executes ABORT

ABORT: $\mathcal{S}$ sends $tx^{\text{refund},0,i}$ with $\sigma^{\text{refund},0,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},0,i}, y^{\text{refund}})$ to $\mathcal{F}^{\text{swap}}$

TRANSFER₁: After all coins are locked, $\mathcal{S}$ sends $tx^{10,j}$ with $\sigma^{10,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{10,j}, y^{\text{transfer}})$ to $\mathcal{F}^{\text{swap}}$

REFUND: $\mathcal{S}$ sends $tx^{\text{refund},0,i}$ with $\sigma^{\text{refund},0,i}$ for a coin not redeemed by $\mathcal{A}$ to $\mathcal{F}^{\text{swap}}$

TRANSFER₀: After T^{transfer} , $\mathcal{S}$ sends $tx^{\text{transfer},T,j}$ with $\sigma^{\text{transfer},T,j}$ to $\mathcal{F}^{\text{swap}}$

Case 3. P_1 is honest and P_0 is malicious. As in case 2, we describe the interaction of the simulator $\mathcal{S}$ with the adversary $\mathcal{A}$ and the ideal functionality. $\mathcal{S}$ is initialized with $V_1, PK_1^{\text{wallet}}, SK_1^{\text{wallet}}$ and $\mathcal{A}$ with $V_0, PK_0^{\text{wallet}}, SK_0^{\text{wallet}}$, and both agree on $T^{\text{refund}}, T^{\text{transfer}}$ and the coins to be swapped.

LOCK: • $\mathcal{S}$ gets Y^{refund} from $\mathcal{A}$

- $\mathcal{S}$ and $\mathcal{A}$ run $(pk^{01,i}, sk_0^{01,i}, sk_1^{01,i}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_1^{01,i}$ and $\mathcal{A}$ gets $sk_0^{01,i}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{\text{refund},0,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}})$
- $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{01,i}, tx^{\text{refund},0,i}, Y^{\text{refund}}, \tilde{\sigma}^{\text{refund},0,i})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ waits until $\mathcal{F}^{\text{swap}}$ has posted all n $tx_0^{\text{lock},i}$ with $\sigma_0^{\text{lock},i}$ from $\mathcal{A}$ and executes ABORT if the lock addresses differ from $pk^{01,i}$
- $\mathcal{S}$ and $\mathcal{A}$ run $(pk^{10,j}, sk_0^{10,j}, sk_1^{10,j}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_1^{10,j}$ and $\mathcal{A}$ gets $sk_0^{10,j}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\sigma^{\text{refund},1,T,j} \leftarrow \text{MuSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{\text{refund},1,T,j})$
- $\mathcal{S}$ checks $1 = \text{MuVrfy}(pk^{10,j}, tx^{\text{refund},1,T,j}, \sigma^{\text{refund},1,T,j})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ sends $tx_1^{\text{lock},j}$ with $\sigma_1^{\text{lock},j} \leftarrow \text{Sign}(sk_1^{\text{wallet},j}, tx_1^{\text{lock},j})$ to $\mathcal{F}^{\text{swap}}$
- Run $(pk^{\text{delay},j}, sk_0^{\text{delay},j}, sk_1^{\text{delay},j}) \leftarrow \text{MuKGen}(1^\lambda)$. $\mathcal{S}$ gets $sk_1^{\text{delay},j}$ and $\mathcal{A}$ gets $sk_0^{\text{delay},j}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{\text{refund},1,j} \leftarrow \text{MuPSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{refund},1,j}, Y^{\text{refund}})$
- $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{\text{delay},j}, tx^{\text{refund},1,j}, Y^{\text{refund}}, \tilde{\sigma}^{\text{refund},1,j})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ gets Y^{transfer} from $\mathcal{A}$
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{01,i} \leftarrow \text{MuPSign}(sk_0^{01,i}, sk_1^{01,i}, tx^{01,i}, Y^{\text{transfer}})$
- $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{01,i}, tx^{01,i}, Y^{\text{transfer}}, \tilde{\sigma}^{01,i})$; else $\mathcal{S}$ executes ABORT
- $\mathcal{S}$ and $\mathcal{A}$ run $\tilde{\sigma}^{10,j} \leftarrow \text{MuPSign}(sk_0^{10,j}, sk_1^{10,j}, tx^{10,j}, Y^{\text{transfer}})$
- $\mathcal{S}$ checks $1 = \text{MuPVrfy}(pk^{10,j}, tx^{10,j}, Y^{\text{transfer}}, \tilde{\sigma}^{10,j})$; else $\mathcal{S}$ executes ABORT

- $\mathcal{S}$ and $\mathcal{A}$ run $\sigma^{\text{transfer},T,j} \leftarrow \text{MuSign}(sk_0^{\text{delay},j}, sk_1^{\text{delay},j}, tx^{\text{transfer},T,j})$

ABORT: After T^{refund} , $\mathcal{S}$ sends $tx^{\text{refund},1,T,j}$ with $\sigma^{\text{refund},1,T,j}$ to $\mathcal{F}^{\text{swap}}$

TRANSFER₁: After the first commit transaction with $\sigma^{10,j}$ has been posted, $\mathcal{S}$ sends $tx^{01,i}$ with $\sigma^{01,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{01,i}, y^{\text{transfer}} \leftarrow \text{MuExt}(\sigma^{10,j}, \tilde{\sigma}^{10,j}, Y^{\text{transfer}}))$ to $\mathcal{F}^{\text{swap}}$

REFUND: After $\mathcal{A}$'s refund transaction with $\sigma^{\text{refund},0,i}$ has been posted, $\mathcal{S}$ sends either the timed refund transactions $tx^{\text{refund},1,T,j}$ with $\sigma^{\text{refund},1,T,j}$ or the immediate refund transactions $tx^{\text{refund},1,j}$ with $\sigma^{\text{refund},1,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},1,j}, y^{\text{refund}} \leftarrow \text{MuExt}(\sigma^{\text{refund},0,i}, \tilde{\sigma}^{\text{refund},0,i}, Y^{\text{refund}}))$ (depending on whether v_1^j has been committed yet or not) to $\mathcal{F}^{\text{swap}}$

TRANSFER₀: Neither $\mathcal{F}^{\text{swap}}$ nor $\mathcal{A}$ expect anything from $\mathcal{S}$ in this phase

We now proceed to argue the indistinguishability between these simulators (in combination with $\mathcal{F}^{\text{swap}}$) and the real protocol. The information $\mathcal{A}$ gets is the same in the real world and in the ideal world, because $\mathcal{S}$ simulates the communication with the other party and $\mathcal{F}^{\text{swap}}$ represents the blockchain. However, we also need to show that the transactions that the parties can post are the same in both cases. For this, we reduce the proof for each possible transaction to the underlying cryptographic assumptions as follows: P_0 's ability to refund anytime and to post commit transactions after all coins are locked stems from the *adaptability* (Definition 10) of two-party adaptor signatures. P_1 's ability to refund after T^{refund} and P_0 's ability to claim P_1 's coins after T^{transfer} results from the functionality of timed transactions (Definition 15). After P_0 's commit transaction, the *aEUF-CMA-security* (Definition 9), *extractability* (Definition 11), *adaptability* (Definition 10), and *pre-verify soundness* (Definition 13) of two-party adaptor signatures guarantee that P_1 can redeem P_0 's coins and/or refund P_1 's own coins if P_0 also refunds.

Refund P_0 : P_0 is allowed to refund anytime in the ideal functionality. The real protocol also guarantees this if the adaptor signatures fulfill *adaptability*, which states that a valid pre-signature can be adapted into a valid signature with the corresponding witness. As P_0 has verified pre-signatures $\tilde{\sigma}^{\text{refund},0,i}$ and the witness y^{refund} , this enables P_0 to create valid signatures $\sigma^{\text{refund},0,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},0,i}, y^{\text{refund}})$ for their refund transactions.

Timed refund P_1 : P_1 can refund after T^{refund} according to the ideal functionality. As both parties sign the timed refund transactions $tx^{\text{refund},1,T,j}$ before P_1 locks their coins (and P_1 verifies these signatures), P_1 gets their coins back after T^{refund} in the real protocol, too.

Commit $P_1 \rightarrow P_0$: The ideal functionality specifies that P_0 only posts a commit transaction $tx^{10,j}$ after all coins are locked. This coincides with the real protocol where both parties have to cooperate to create the pre-signatures $\tilde{\sigma}^{10,j}$, which they only do after all coins are locked. Once completed, P_0 can obtain valid signatures for the commit transactions as long as the adaptor signatures satisfy *adaptability*: Since P_0 knows the witness y^{transfer} and possesses verified pre-signatures $\tilde{\sigma}^{10,j}$, P_0 can calculate $\sigma^{10,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{10,j}, y^{\text{transfer}})$.

Transfer $P_0 \rightarrow P_1$: The ideal functionality only allows P_1 to redeem P_0 's coins after P_0 has posted a commit transaction $tx^{10,j}$. In the real protocol, this is also given if the adaptor signatures fulfill *aEUF-CMA-security*, which ensures that no adversary can forge a valid signature on a message even if given a pre-signature on that message. Therefore, P_1 cannot produce a valid signature $\sigma^{01,i}$ just from the pre-signature $\tilde{\sigma}^{01,i}$. The ability to do so after P_0 's first commit transaction stems from the combination of *extractability*, *adaptability* and *pre-verify soundness* of two-party adaptor signatures. First, *extractability* guarantees that it is possible to extract the witness from a pre-signature and a valid signature on the same message. When P_1 sees the signature $\sigma^{10,j}$ on P_0 's commit transaction, P_1 can use this to obtain the witness y^{transfer} from the previously created pre-signature $\tilde{\sigma}^{10,j}$: $y^{\text{transfer}} \leftarrow \text{MuExt}(\sigma^{10,j}, \tilde{\sigma}^{10,j}, Y^{\text{transfer}})$. Then, *adaptability* makes sure that P_1 can use this witness and the verified pre-signatures $\tilde{\sigma}^{01,i}$ to create valid signatures $\sigma^{01,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{01,i}, y^{\text{transfer}})$ for the redeem transactions. Finally, *pre-verify*

soundness ensures that a statement which is not part of the hard relation R cannot produce a verifiable pre-signature. Therefore, if P_0 had sent P_1 a faulty statement Y^{transfer} , the pre-signatures $\tilde{\sigma}^{01,i}$ would not have been valid and P_1 would have aborted before P_0 could commit.

Refund P_1 : Similarly, once P_0 has committed, the ideal functionality only allows P_1 to refund their coins after P_0 has posted a refund transaction $tx^{\text{refund},0,i}$. Again, this is also given in the real protocol if the adaptor signatures meet *aEUF-CMA-security*, as this ensures that P_1 cannot produce a valid signature $\sigma^{\text{refund},1,j}$ just from the pre-signature $\tilde{\sigma}^{\text{refund},1,j}$. The ability to do so after P_0 's first refund transaction stems from the combination of *extractability*, *adaptability* and *pre-verify soundness* of two-party adaptor signatures. First, *extractability* guarantees that P_1 can extract the witness y^{refund} from the observed signature $\sigma^{\text{refund},0,i}$ on P_0 's refund transaction and the previously created pre-signature $\tilde{\sigma}^{\text{refund},0,i}$: $y^{\text{refund}} \leftarrow \text{MuExt}(\sigma^{\text{refund},0,i}, \tilde{\sigma}^{\text{refund},0,i}, Y^{\text{refund}})$. Then, *adaptability* makes sure that P_1 can use this witness and the verified pre-signatures $\tilde{\sigma}^{\text{refund},1,j}$ to create valid signatures $\sigma^{\text{refund},1,j} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{\text{refund},1,j}, y^{\text{refund}})$ for P_1 's refund transactions. Finally, *pre-verify soundness* ensures that if P_0 had sent P_1 a faulty statement $(\cdot, Y^{\text{refund}}) \notin R$, the pre-signatures $\tilde{\sigma}^{\text{refund},1,j}$ would not have been valid and P_1 would have aborted before P_0 could commit.

Transfer $P_1 \rightarrow P_0$: P_0 can claim P_1 's coins after T^{transfer} according to the ideal functionality. As both parties sign these timed transfer transactions $tx^{\text{transfer},T,j}$ before P_1 can claim P_0 's coins (and P_0 verifies these signatures), P_0 gets P_1 's coins after T^{transfer} in the real protocol, too.

Remark 4. In general, no other than the pre-defined transactions are possible as the block-chain signatures have to fulfill EUF-CMA-security (Definition 2) for one-party signatures and EUF-CMA-security (Definition 4) for two-party signatures, so no signatures can be forged. The only exception are the lock transactions, as the parties know the secret keys of their wallets and can therefore transfer their coins anywhere they wish. This is not a problem however, as in this case the other party simply aborts both in the real world and in the ideal world.

We conclude that the protocol Π_{Swap} UC-emulates the ideal functionality $\mathcal{F}^{\text{swap}}$ according to Definition 20 and therefore, by extension, satisfies the security requirements of Definition 16. $\square$

A Note on Rogue Key Attacks. When generating a joint key, one always has to be aware of so-called rogue key attacks [RY07]. We can exemplify this notion by considering Schnorr [Sch90] signatures: Here, the public key is simply the aggregate of both parties' personal public keys, i.e. $pk = pk_0 + pk_1$. Now, let us assume that P_0 is malicious and P_1 is honest. To generate the joint key, P_1 picks their keys (sk_1, pk_1) and faithfully sends the public key pk_1 to P_0 . Following this, P_0 picks their keys (sk_0, pk_0) , but instead of sending their public key pk_0 to P_1 , P_0 sends $pk_0^* = pk_0 - pk_1$. The problem is now that the joint public key, as the aggregate of both parties' personal public keys, yields $pk_0^* + pk_1 = pk_0 - pk_1 + pk_1 = pk_0$. Therefore, P_0 is in possession of the joint secret key rather than just knowing a part of the secret key. If coins were moved to this joint address, P_0 would have full control over their spending. We prevent this problem in our protocol by relying on the security requirements for two-party signatures in Definition 4: The EUF-CMA-security guarantees that an adversary cannot forge a signature (except with negligible probability) even when it corrupts one party and is able to choose their public key. This EUF-CMA-security would be violated by a two-party signature scheme that allows for rogue key attacks, because with a rogue public key, the adversary could actually produce a forged signature with the joint secret key that it owns.

3.4 Privacy Analysis

We now formally analyze and prove the protocol's privacy properties, namely unlinkability and fungibility.

Theorem 3 (Privacy of Π_{Swap}). *The protocol Π_{Swap} meets the privacy requirements of unlinkability and fungibility according to Definitions 17 and 18.*

Proof. We prove unlinkability by constructing a reduction $\mathcal{R}$ that uses an adversary $\mathcal{A}$ breaking the unlinkability of atomic swaps to break the *unlinkability* of adaptor signatures as stated in Definition 12. First, we prove that no PPT adversary $\mathcal{A}$ can link n -to-1 atomic swaps. Then we extend this argument to n -to- m atomic swaps using a hybrid argument. In doing so, we omit the lock and timed transactions as they are completely independent on each blockchain and instead concentrate on the transfer (or refund) transactions where the signatures are created with adaptor signatures using a common witness/statement pair.

Reduction $\mathcal{R}$ receives a public key pk and has access to oracles $\mathcal{O}_{\text{MuSign}}$ and $\mathcal{O}_{\text{MuPSign}}$. First, $\mathcal{R}$ generates its own witness/statement pair $(y^*, Y^*) \in R$ and n public key addresses on possibly disjoint blockchains A with corresponding secret keys $(pk^{A,i}, sk_1^{A,i}, sk_2^{A,i}) \leftarrow \text{MuKGen}(1^\lambda)$, $1 \leq i \leq n$. Reduction $\mathcal{R}$ also chooses corresponding target public key addresses $pk_{\text{transfer}}^{A,i}$ and coins $v^{A,i}$ on these blockchains to create transactions $tx^{A,i} = (pk^{A,i}, pk_{\text{transfer}}^{A,i}, v^{A,i})$. Then, $\mathcal{R}$ pre-signs these transactions, getting adaptor signatures $\tilde{\sigma}^{A,i} \leftarrow \text{MuPSign}(sk_1^{A,i}, sk_2^{A,i}, tx^{A,i}, Y^*)$ and adapts these pre-signatures into regular signatures: $\sigma^{A,i} \leftarrow \text{MuAdapt}(\tilde{\sigma}^{A,i}, y^*)$. This setup perfectly emulates the transactions with their signatures on blockchains A . Finally, $\mathcal{R}$ chooses one coin v^B and a target address pk_{transfer}^B for blockchain B , which $\mathcal{R}$ uses to create a transaction $tx^B = (pk, pk_{\text{transfer}}^B, v^B)$.

$\mathcal{R}$ then submits this transaction tx^B as the challenge message, together with (y^*, Y^*) and gets back a challenge signature σ_b . It proceeds to forward this challenge signature σ_b with transaction tx^B and the transactions $tx^{A,i}$ with their signatures $\sigma^{A,i}$ to the adversary $\mathcal{A}$. It simulates $\mathcal{A}$'s oracle $\mathcal{O}_{\text{Swap}}$ for each query by generating a different witness/statement pair $(y, Y) \in R$ and creating n transactions on blockchains A , pre-signing these transactions with Y and then adapting the pre-signatures into regular signatures with y . Analogous, it generates a transaction on blockchain B , pre-signs this transaction with Y , adapts this pre-signature into a regular signature with y and finally sends all these transactions with their signatures to adversary $\mathcal{A}$. We assume that $\mathcal{A}$ can detect a link between the transactions on blockchains A and B with probability ϵ . Adversary $\mathcal{A}$ outputs $b' = 1$ if $\mathcal{R}$'s challenger creates σ_b with $\text{MuPSign}, \text{MuAdapt}$ ($b = 1$), and $b' = 0$ if it creates σ_b with MuSign ($b = 0$), with non-negligible probability. Reduction $\mathcal{R}$ outputs the same bit it receives from the adversary. Consequently, the success probability ϵ of adversary $\mathcal{A}$ directly transfers to the success probability of $\mathcal{R}$. The assumption of ϵ being non-negligible contradicts our unlinkability assumption of adaptor signatures. Thus, the probability of any PPT adversary $\mathcal{A}$ against our protocol in the case of n -to-1 atomic swaps breaking unlinkability must be negligible.

We now sketch the argument for unlinkability in the generalized case of n -to- m atomic swaps. We define the extreme hybrids "all m transaction signatures on blockchains B are created using MuSign " and "all m transaction signatures on blockchains B are created using MuPSign and MuAdapt ". We then define the hybrid for every step j as "first j transactions $1, \dots, j$ are created using MuSign and the remaining transactions $j+1, \dots, m$ are created with MuPSign and MuAdapt ". At any step j , a distinguisher $\mathcal{D}$ (that wants to distinguish between the extreme hybrids) has to tell the hybrids j and $j+1$ apart, but this is exactly the same as trying to link a n -to-1 atomic swap, which we proved impossible. Therefore, as the change of one item is indistinguishable, such a distinguisher $\mathcal{D}$ cannot exist, proving unlinkability of n -to- m atomic swaps.

Finally, we consider fungibility. All transactions in our protocol have the same form as normal transactions in that they denote an original address, a destination address, the coin, and possibly an execution time. The use of timed transactions is not limited to atomic swaps, they are for example also used for payment plans or conveniently scheduling transactions and then going offline. Similarly, joint outputs are commonly used for sharing assets or improving security outside of atomic swaps, so it is not possible to detect an atomic swap from the transactions used in our scheme. Furthermore, the *unlinkability* of adaptor signatures (see Definition 12) guarantees that it is impossible to distinguish between a normal signature and a signature created with an adaptor signature. Hence, the signatures posted with the transactions in our scheme are indistinguishable from normal transaction signatures. Protocol Π_{Swap} therefore meets the unlinkability and fungibility requirements according to Definitions 17 and 18. $\square$

References

- [Abf21] Jakob Abfalter. *Adaptor signature based atomic swaps between bitcoin and a mumblewimble based cryptocurrency*. eng. Wien, 2021. DOI: [10.34726/hss.2021.77663](https://doi.org/10.34726/hss.2021.77663).
- [AK25] Prerna Arote and Joy Kuri. “CommTLC: An Alternative to Reduce the Attack Surface of HTLCs in Lightning Networks”. In: *IET Blockchain* 5.1 (2025). DOI: [10.1049/BLC2.70005](https://doi.org/10.1049/BLC2.70005). URL: <https://doi.org/10.1049/blc2.70005>.
- [Ame05] American National Standards Institute, Inc. *ANSI X9.62 Public Key Cryptography for the Financial Services Industry: the Elliptic Curve Digital Signature Algorithm (ECDSA)*. Nov. 2005. URL: <https://standards.globalspec.com/std/1955141/ANSI%20X9.62>.
- [Aum+21] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. “Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures”. In: *ASIACRYPT 2021, Part II*. Ed. by Mehdi Tibouchi and Huaxiong Wang. Vol. 13091. LNCS. Springer, Cham, Dec. 2021, pp. 635–664. DOI: [10.1007/978-3-030-92075-3_22](https://doi.org/10.1007/978-3-030-92075-3_22).
- [Bad+20] Christian Badertscher, Ran Canetti, Julia Hesse, Björn Tackmann, and Vassilis Zikas. “Universal Composition with Global Subroutines: Capturing Global Setup Within Plain UC”. In: *TCC 2020, Part III*. Ed. by Rafael Pass and Krzysztof Pietrzak. Vol. 12552. LNCS. Springer, Cham, Nov. 2020, pp. 1–30. DOI: [10.1007/978-3-030-64381-2_1](https://doi.org/10.1007/978-3-030-64381-2_1).
- [Bao+23] Zijian Bao, Debiao He, Cong Peng, Min Luo, and Kim-Kwang Raymond Choo. “An Identity-Based Adaptor Signature Scheme and its Applications in the Blockchain System”. In: *IEEE Open J. Comput. Soc.* 4 (2023), pp. 231–242. DOI: [10.1109/OJCS.2023.3309836](https://doi.org/10.1109/OJCS.2023.3309836). URL: <https://doi.org/10.1109/OJCS.2023.3309836>.
- [BDN18] Dan Boneh, Manu Drijvers, and Gregory Neven. *Compact Multi-Signatures for Smaller Blockchains*. Cryptology ePrint Archive, Report 2018/483. 2018. URL: <https://eprint.iacr.org/2018/483>.
- [Bol03] Alexandra Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *PKC 2003*. Ed. by Yvo Desmedt. Vol. 2567. LNCS. Springer, Berlin, Heidelberg, Jan. 2003, pp. 31–46. DOI: [10.1007/3-540-36288-6_3](https://doi.org/10.1007/3-540-36288-6_3).

- [BPW23] Dan Boneh, Aditi Partap, and Brent Waters. *Accountable Multi-Signatures with Constant Size Public Keys*. Cryptology ePrint Archive, Report 2023/1793. 2023. URL: <https://eprint.iacr.org/2023/1793>.
- [But13] Vitalik Buterin. *Ethereum White Paper: A Next Generation Smart Contract & Decentralized Application Platform*. 2013. URL: https://cryptorating.eu/whitepapers/Ethereum/Ethereum_white_paper.pdf.
- [BW24] Renas Bacho and Benedikt Wagner. “Tightly Secure Non-interactive BLS Multi-signatures”. In: *ASIACRYPT 2024, Part II*. Ed. by Kai-Min Chung and Yu Sasaki. Vol. 15485. LNCS. Springer, Singapore, Dec. 2024, pp. 397–422. DOI: [10.1007/978-981-96-0888-1_13](https://doi.org/10.1007/978-981-96-0888-1_13).
- [Can01] Ran Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols”. In: *42nd FOCS*. IEEE Computer Society Press, Oct. 2001, pp. 136–145. DOI: [10.1109/SFCS.2001.959888](https://doi.org/10.1109/SFCS.2001.959888).
- [Can20] Ran Canetti. “Universally Composable Security”. In: *J. ACM* 67.5 (2020), 28:1–28:94. DOI: [10.1145/3402457](https://doi.org/10.1145/3402457). URL: <https://doi.org/10.1145/3402457>.
- [Cla+24] Emily Clark, Chloe Georgiou, Katelyn Poon, and Marek Chrobak. “On HTLC-Based Protocols for Multi-Party Cross-Chain Swaps”. In: *35th International Symposium on Algorithms and Computation, ISAAC 2024, December 8-11, 2024, Sydney, Australia*. Ed. by Julián Mestre and Anthony Wirth. Vol. 322. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 22:1–22:15. DOI: [10.4230/LIPICS.ISAAC.2024.22](https://doi.org/10.4230/LIPICS.ISAAC.2024.22). URL: <https://doi.org/10.4230/LIPIcs.ISAAC.2024.22>.
- [DH20] Apoorvaa Deshpande and Maurice Herlihy. “Privacy-Preserving Cross-Chain Atomic Swaps”. In: *FC 2020 Workshops*. Ed. by Matthew Bernhard, Andrea Bracciali, L. Jean Camp, Shin’ichiro Matsuo, Alana Maurushat, Peter B. Rønne, and Massimiliano Sala. Vol. 12063. LNCS. Springer, Cham, Feb. 2020, pp. 540–549. DOI: [10.1007/978-3-030-54455-3_38](https://doi.org/10.1007/978-3-030-54455-3_38).
- [DN14] Ivan Damgård and Jesper Buus Nielsen. “Adaptive versus Static Security in the UC Model”. In: *ProvSec 2014*. Ed. by Sherman S. M. Chow, Joseph K. Liu, Lucas C. K. Hui, and Siu-Ming Yiu. Vol. 8782. LNCS. Springer, Cham, Oct. 2014, pp. 10–28. DOI: [10.1007/978-3-319-12475-9_2](https://doi.org/10.1007/978-3-319-12475-9_2).
- [DOY22] Wei Dai, Tatsuki Okamoto, and Go Yamamoto. *Stronger Security and Generic Constructions for Adaptor Signatures*. Cryptology ePrint Archive, Report 2022/1687. 2022. URL: <https://eprint.iacr.org/2022/1687>.
- [Duc+18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme”. In: *IACR TCHES* 2018.1 (2018), pp. 238–268. ISSN: 2569-2925. DOI: [10.13154/tches.v2018.i1.238-268](https://tches.iacr.org/index.php/TCHES/article/view/839). URL: <https://tches.iacr.org/index.php/TCHES/article/view/839>.
- [Erw+21] Andreas Erwig, Sebastian Faust, Kristina Hostáková, Monosij Maitra, and Siavash Riahi. *Two-Party Adaptor Signatures From Identification Schemes*. Cryptology ePrint Archive, Report 2021/150. 2021. URL: <https://eprint.iacr.org/2021/150>.
- [Ger+24] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. “Foundations of Adaptor Signatures”. In: *EUROCRYPT 2024, Part II*. Ed. by Marc Joye and Gregor Leander. Vol. 14652. LNCS. Springer, Cham, May 2024, pp. 161–189. DOI: [10.1007/978-3-031-58723-8_6](https://doi.org/10.1007/978-3-031-58723-8_6).

- [gma12] gmaxwell. *P2PTradeX: P2P Trading between cryptocurrencies*. 2012. URL: <https://bitcointalk.org/index.php?topic=91843.msg1011956#msg1011956>.
- [GRS25] Paul Gerhart, Daniel Rausch, and Dominique Schröder. “Universally Composable Adaptor Signatures”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 1363. URL: <https://eprint.iacr.org/2025/1363>.
- [Gug20] Joël Gugger. *Bitcoin-Monero Cross-chain Atomic Swap*. Cryptology ePrint Archive, Report 2020/1126. 2020. URL: <https://eprint.iacr.org/2020/1126>.
- [Her18] Maurice Herlihy. “Atomic Cross-Chain Swaps”. In: *37th ACM PODC*. Ed. by Calvin Newport and Idit Keidar. ACM, July 2018, pp. 245–254. DOI: [10.1145/3212734.3212736](https://doi.org/10.1145/3212734.3212736).
- [Hoe+22] Philipp Hoenisch, Subhra Mazumdar, Pedro Moreno-Sanchez, and Sushmita Ruj. “LightSwap: An Atomic Swap Does Not Require Timeouts at both Blockchains”. In: *Data Privacy Management, Cryptocurrencies and Blockchain Technology: ESORICS 2022 International Workshops, DPM 2022 and CBT 2022, Copenhagen, Denmark, September 26–30, 2022, Revised Selected Papers*. Copenhagen, Denmark: Springer-Verlag, 2022, pp. 219–235. ISBN: 978-3-031-25733-9. DOI: [10.1007/978-3-031-25734-6_14](https://doi.org/10.1007/978-3-031-25734-6_14). URL: https://doi.org/10.1007/978-3-031-25734-6_14.
- [Ji+24] Yunfeng Ji, Yuting Xiao, Birou Gao, and Rui Zhang. “Threshold/Multi Adaptor Signature and Their Applications in Blockchains”. In: *Electronics* 13.1 (2024). ISSN: 2079-9292. DOI: [10.3390/electronics13010076](https://doi.org/10.3390/electronics13010076).
- [Kar+24] Ioanna Karantaidou, Omar Renawi, Foteini Baldimtsi, Nikolaos Kamarinakis, Jonathan Katz, and Julian Loss. “Blind Multisignatures for Anonymous Tokens with Decentralized Issuance”. In: *ACM CCS 2024*. Ed. by Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie. ACM Press, Oct. 2024, pp. 1508–1522. DOI: [10.1145/3658644.3690364](https://doi.org/10.1145/3658644.3690364).
- [Lee11] Charlie Lee. *Litecoin*. Oct. 2011. URL: <https://github.com/litecoin-project/litecoin>.
- [Li+24] Tao Li, Peiyao Niu, Yilei Wang, Shengke Zeng, Xiaoying Wang, and Willy Susilo. “HT2REP: A fair cross-chain atomic exchange protocol under UC framework based on HTLCs and TRE”. In: *Comput. Stand. Interfaces* 89 (2024), p. 103834. DOI: [10.1016/j.csi.2024.103834](https://doi.org/10.1016/j.csi.2024.103834). URL: <https://doi.org/10.1016/j.csi.2024.103834>.
- [LW20] Jan Ladleif and Mathias Weske. “Time in Blockchain-Based Process Execution”. In: *24th IEEE International Enterprise Distributed Object Computing Conference, EDOC 2020, Eindhoven, The Netherlands, October 5-8, 2020*. IEEE, 2020, pp. 217–226. DOI: [10.1109/EDOC49727.2020.00034](https://doi.org/10.1109/EDOC49727.2020.00034). URL: <https://doi.org/10.1109/EDOC49727.2020.00034>.
- [Maz22] Subhra Mazumdar. “Towards faster settlement in HTLC-based Cross-Chain Atomic Swaps”. In: *4th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications, TPS-ISA 2022, Atlanta, GA, USA, December 14-17, 2022*. IEEE, 2022, pp. 295–304. DOI: [10.1109/TPS-ISA56441.2022.00043](https://doi.org/10.1109/TPS-ISA56441.2022.00043). URL: <https://doi.org/10.1109/TPS-ISA56441.2022.00043>.
- [Nak08] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: <https://bitcoin.org/bitcoin.pdf>.

- [PD16] Joseph Poon and Thaddeus Dryja. *The bitcoin lightning network: Scalable off-chain instant payments*. 2016. URL: <https://lightning.network/lightning-network-paper.pdf>.
- [Poe17] Andrew Poelstra. *Scriptless Scripts*. 2017. URL: <https://download.wpsoftware.net/bitcoin/wizardry/mw-slides/2017-03-mit-bitcoin-expo/slides.pdf>.
- [RY07] Thomas Ristenpart and Scott Yilek. *The Power of Proofs-of-Possession: Securing Multiparty Signatures against Rogue-Key Attacks*. Cryptology ePrint Archive, Report 2007/264. 2007. URL: <https://eprint.iacr.org/2007/264>.
- [Sch90] Claus-Peter Schnorr. “Efficient Identification and Signatures for Smart Cards”. In: *CRYPTO’89*. Ed. by Gilles Brassard. Vol. 435. LNCS. Springer, New York, Aug. 1990, pp. 239–252. DOI: [10.1007/0-387-34805-0_22](https://doi.org/10.1007/0-387-34805-0_22).
- [Tie13] TierNolan. *Alt chains and atomic transfers*. 2013. URL: <https://bitcointalk.org/index.php?topic=193281.msg2224949#msg2224949>.
- [TMM22] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. “Universal Atomic Swaps: Secure Exchange of Coins Across All Blockchains”. In: *2022 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2022, pp. 1299–1316. DOI: [10.1109/SP46214.2022.9833731](https://doi.org/10.1109/SP46214.2022.9833731).
- [TS15] Stefan Thomas and Evan Schwartz. “A protocol for interledger payments”. In: *URL https://interledger.org/interledger.pdf* (2015), p. 45.
- [TZ23] Guofeng Tang and Zhenfeng Zhang. “Two-Party Signing For ISO/IEC Digital Signature Standards”. In: *Comput. J.* 66.5 (2023), pp. 1111–1125. DOI: [10.1093/COMJNL/BXAC001](https://doi.org/10.1093/COMJNL/BXAC001). URL: <https://doi.org/10.1093/comjnl/bxac001>.
- [TZY22] Binbin Tu, Min Zhang, and Chen Yu. “Efficient ECDSA-Based Adaptor Signature for Batched Atomic Swaps”. In: *ISC 2022*. Ed. by Willy Susilo, Xiaofeng Chen, Fuchun Guo, Yudi Zhang, and Rolly Intan. Vol. 13640. LNCS. Springer, Cham, Dec. 2022, pp. 175–193. DOI: [10.1007/978-3-031-22390-7_12](https://doi.org/10.1007/978-3-031-22390-7_12).
- [Wad+22] Sarisht Wadhwa, Jannis Stoeter, Fan Zhang, and Kartik Nayak. *He-HTLC: Revisiting Incentives in HTLC*. Cryptology ePrint Archive, Report 2022/546. 2022. URL: <https://eprint.iacr.org/2022/546>.
- [XAD21] Jiahua Xu, Damien Ackerer, and Alevtina Dubovitskaya. “A Game-Theoretic Analysis of Cross-Chain Atomic Swaps with HTLCs”. In: *41st IEEE International Conference on Distributed Computing Systems, ICDCS 2021, Washington DC, USA, July 7-10, 2021*. IEEE, 2021, pp. 584–594. DOI: [10.1109/ICDCS51616.2021.00062](https://doi.org/10.1109/ICDCS51616.2021.00062). URL: <https://doi.org/10.1109/ICDCS51616.2021.00062>.
- [Yak17] Anatoly Yakovenko. *Solana: A new architecture for a high performance blockchain*. 2017. URL: <https://solana.com/solana-whitepaper.pdf>.
- [Yoc17] Jake Yocom-Piatt. *On-Chain Atomic Swaps*. Sept. 2017. URL: <https://blog.decred.org/2017/09/20/On-Chain-Atomic-Swaps/>.
- [Zhu+23] Xinjie Zhu, Debiao He, Zijian Bao, Cong Peng, and Min Luo. “Two-Party Adaptor Signature Scheme Based on IEEE P1363 Identity-Based Signature”. In: *IEEE Open J. Commun. Soc.* 4 (2023), pp. 2717–2728. DOI: [10.1109/OJCOMS.2023.3325106](https://doi.org/10.1109/OJCOMS.2023.3325106). URL: <https://doi.org/10.1109/OJCOMS.2023.3325106>.